

面向对象程序设计

第 1 讲 Java 简介

刘进

2230652597@qq.com

OOP 教辅 2025 秋季上午 QQ 群：

837966056





教材、参考书

- 教材

➢ [1]Java 语言程序设计，沈泽刚 编，清华大学出版社，2023-06-01 ， ISBN ： 9787302629726 。



- 参考书籍

➢ [1] Java 语言程序设计 : 基础篇 (原书第 12 版), (美) 梁勇 (Y. ， Daniel ， Liang) 著，戴开宇译 , 机械工业出版社 , 2021.5, ISBN ： 9787111669807

➢ [2] Java 语言程序设计 (第 3 版), 郑莉 , 张宇著 , 清华大学出版社 ,2021.8,ISBN ： 9787302581659

➢ [3]Java 核心技术卷 I: 基础知识 (原书第 11 版),(美) 凯 ·S. 霍斯特曼 (Cay S.Horstmann) 著 , 机械工业出版社 , 2019.12, ISBN ： 9787111636663

➢ [4] 深入理解 Java 虚拟机： JVM 高级特性与最佳实践 (第 3 版), 周志明著 , 机械工业出版社 ,2020.1,ISBN ： 9787111641247

➢ [5]Java 编程思想 (第 4 版), (美) Bruce Eckel 著 ，陈昊鹏译 , 机械工业出版社 ,2007.6, ISBN ： 9787111213826

教学安排

主要内容

9/ 至 12/	1	第 1 讲 Java 程序设计简介 1.1 Java 发展历程 1.2 Java 环境搭建 1.3 Java 程序基本结构 1.4 Eclipse 集成开发环境 第 2 讲 Java 基本语法-关键字&运算符&表达式 2.1 关键字、保留字、标识符
	2	第 2 讲 Java 基本语法-关键字&运算符&表达式 2.2 变量、数据类型和数据类型转换 2.3 运算符 2.4 表达式 第 3 讲 Java 基本语法-控制结构&数组 3.1 流程控制（选择、循环） 3.2 数组 3.3 示例学习
	3	第 4 讲 Java OOP -概述&封装隐藏 4.1 面向对象概述 4.2 面向对象特征之封装与隐藏 第 5 讲 Java OOP -继承&多态 5.1 面向对象特征之继承
	4	第 5 讲 Java OOP -继承&多态 5.2 面向对象特征之多态
	5	第 6 讲 JavaOOP-修饰符接口包装类内部类 6.1 static、final 和 abstract 修饰符 6.2 接口 6.3 类 Object 与包装类 6.4 内部类
	6	第 7 讲 Java OOP-设计模式 7.1 设计模式 7.2 单例设计模式 Singleton 7.3 模板方法设计模式 TemplateMethod 7.4 工厂 FactoryMethod 代理模式 Proxy
	7	第 8 讲 Java OOP-泛型&枚举&注解 第 9 讲 Java OOP 开发编程-集合
	8	第 11 讲 Java OOP 开发编程-多线程 11.1 多线程实现 11.2 多线程同步
	9	第 13 讲 Java OOP 开发编程-IO
	10	第 14 讲 Java OOP 开发编程-访问数据库 14.1 JDBC 介绍 14.2 数据库访问
	11	第 15 讲 Java OOP 开发编程-反射
	12	第 16 讲 Java OOP 开发编程-网络

考核方式

- 闭卷考试

- **课后作业**（ 20% ）+ **课间测验**（ 10% ）+ **期中考试**（ 10% ）
- + 期末笔试（ 60% ）

- 作业和实验提交方式后续在本课程 QQ 群中通知。

- 本课程 QQ 群：
  OOP 教辅 2025 秋季 6 、 7 节　　837966056

第 1 章 Java 程序设计简介

目录

1.1 Java 发展历程

1.2 Java 环境搭建

1.3 Java 程序基本结构

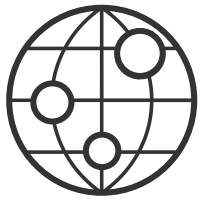
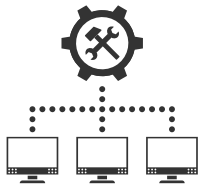
1.4 Eclipse 集成开发环境

Baseline:

1 ）基础： C/C++

2 ）应用： Java

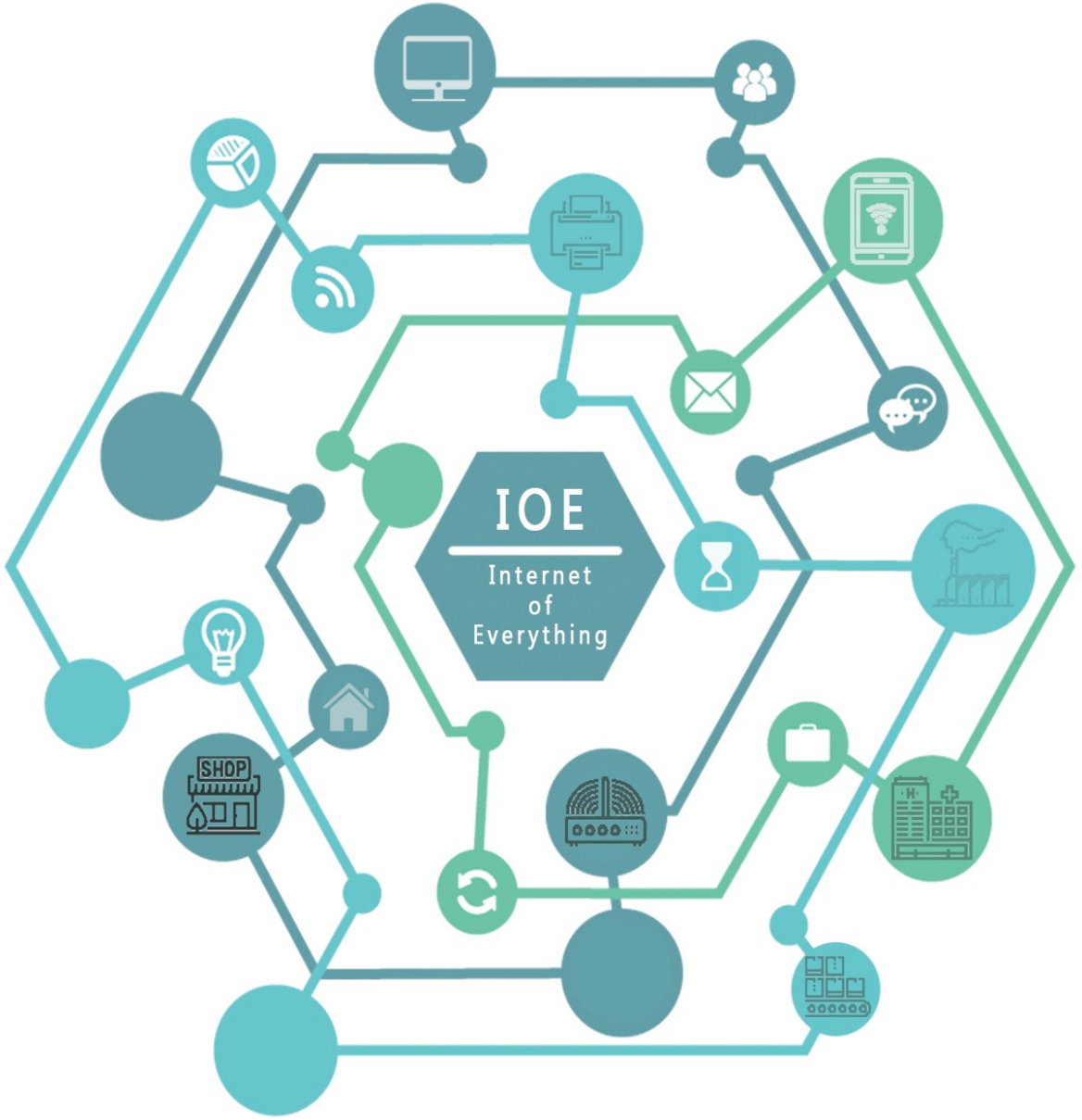


1.1 Java 发展历史

技术是用出来的

Java 来自于 Sun 公司的一个叫 Green 的项目，其原先的目的是为家用消费电子产品开发一个**分布式代码系统**，这样可以将指令通过网络发给电冰箱、电视机等家用电器，对它们进行控制，和它们进行信息交流。开始， **Green** 准备采用 C++ ，但 C++ 太复杂，安全性差，最后基于 C++ 开发了一种新的语言 Oak1991（一种用于网络的精巧而安全的语言）。 很快， Sun 公司将 Oak 改名为 Java ，作为一种**面向 Internet 的编程语言**进军 **Internet** ，并逐渐发展成为 **Web 应用程序的首选开发语言**。



技术的螺旋式上升

1）集中与分散

2）从万物互联到万物智联

Java 发展历史

技术的演化与进化

如何活得更久

- 1995 年 5 月 23 日，Java 语言诞生
- 1996 年 1 月，第一个 JDK-JDK1.0 诞生
- 1999 年 6 月，**SUN 公司**发布 Java 的三个版本：标准版（ JavaSE，以前是 J2SE ）、企业版（ JavaEE 以前是 J2EE ）和微型版（ JavaME ，以前是 J2ME ）
- 2004 年 9 月 30 日 J2SE1.5 发布，成为 Java 语言发展史上的又一里程碑。为了表示该版本的重要性，J2SE1.5 更名为 Java SE 5.0
- 2005 年 6 月，JavaOne 大会召开，SUN 公司公开 Java SE 6 。此时，Java 的各种版本已经更名，以取消其中的数字 "2" ：J2EE 更名为 Java EE ，J2SE 更名为 Java SE ，J2ME 更名为 Java ME
- 2009 年 04 月 20 日，甲骨文 74 亿美元收购 Sun ，取得 Java 的版权。
- 2014 年 3 月 18 日，**Oracle 公司**发表 Java SE 8 。 --- 仍然是最受欢迎的版本
- 2018 年 9 月 26 日 ,Oracle 官方宣布 Java 11 正式发布，这是 Java 大版本周期变化后的第一个长期支持版本。
- 2021 年 9 月 14 日 Java 17 发布，**Java 17 是 Java 11 以来又一个 LTS（长期支持）版本**。

Java 语言的诞生、特点

Java 之父 James Gosling 团队在开发” Green ”项目时，发现 C 缺少**垃圾回收系统，还有可移植的安全性、分布程序设计、和多线程功能**。最后，他们想要一种**易于移植到各种设备上的平台**。

Java 确实是从 C 语言和 C++ 语言继承了许多成份，甚至可以将 Java 看成是**类 C 语言**发展和衍生的产物。比如 Java 语言的变量声明，操作符形式，参数传递，流程控制等方面和 C 语言、C++ 语言完全相同。

但同时， Java 是一个**纯粹的面向对象**的程序设计语言，它继承了 C++ 语言面向对象技术的核心。**Java 舍弃了 C 语言中容易引起错误的指针（以引用取代）、运算符重载（ operator overloading ）、多重继承（以接口取代）等特性，增加了垃圾回收器功能用于回收不再被引用的对象所占据的内存空间。JDK1.5 又引入了泛型编程（ Generic Programming ）、类型安全的枚举、不定长参数和自动装 / 拆箱**



Java 之父 James Gosling

Java 语言优势和特点

“新特性”与语用

Java 语言是面向对象的。 Java 是一种面向对象的语言，它对对象中的类、对象、继承、封装、多态、接口、包等均有很好的支持。Java 语言提供类、接口和继承等原语，为了简单起见，只支持类之间的单继承，但支持接口之间的多继承，并支持类与接口之间的实现机制（关键字为 implements ）。

Java 语言是健壮的。 Java 的强类型机制、异常处理、垃圾的自动收集等是 Java 程序健壮性的重要保证。对指针的丢弃是 Java 的一大进步。

Java 语言是安全的。Java 通常被用在网络环境中，为此，Java 提供了一个安全机制以防恶意代码的攻击。如：安全防范机制（类 ClassLoader ），分配不同的名字空间以防替代本地的同名类、字节代码检查，并包含安全管理机制。

Java 是性能略高的。与那些解释型的高级脚本语言相比，Java 的性能还是较优的。随着 JIT(Just in Time) 的发展，Java 的运行速度也越来越高。

Java 语言是体系结构中立的。 Java 程序（后缀为 java 的文件）在 Java 平台上被编译为体系结构中立的字节码格式（后缀为 class 的文件），然后可以在实现这个 Java 平台的任何系统中运行。

Java 语言是解释型的。如前所述，Java 程序在 Java 平台上被编译为字节码格式，然后可以在实现这个 Java 平台的任何系统的解释器中运行。

Java 语言是多线程的。这也是 Java 语言的一大特性，它必须由 Thread 类和它的子类来创建。 Java 支持多个线程同时执行，并提供多线程之间的同步机制。任何一个线程都有自己的 run() 方法，要执行的方法就写在 run() 方法体内。

Java 语言是分布式的。 Java 语言支持 Internet 应用的开发，在基本的 Java 应用编程接口中就有一个网络应用编程接口（ java net ），它提供了用于网络应用编程的类库，包括 URL 、 URLConnection 、 Socket 等。 Java 的 RMI （远程方法激活）机制也是开发分布式应用的重要手段。

“新特性”与语用

图灵机

打孔卡片

机器代码

汇编语言

Fortran

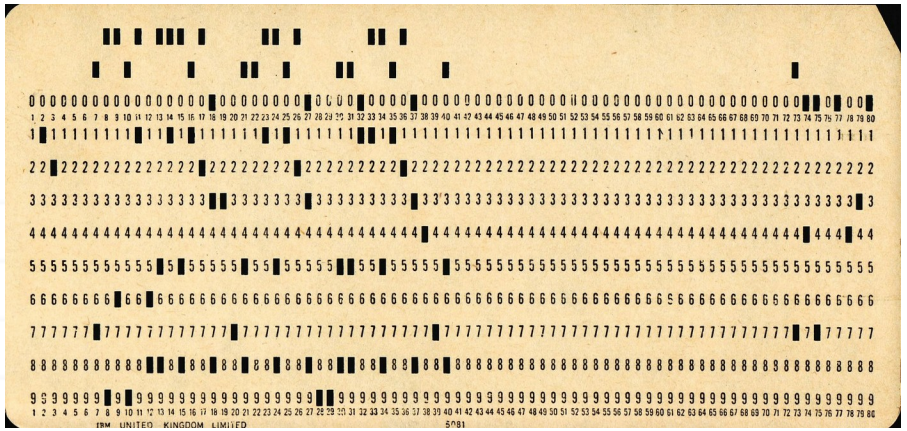
COBOL

ALGOL 6(

Pascal

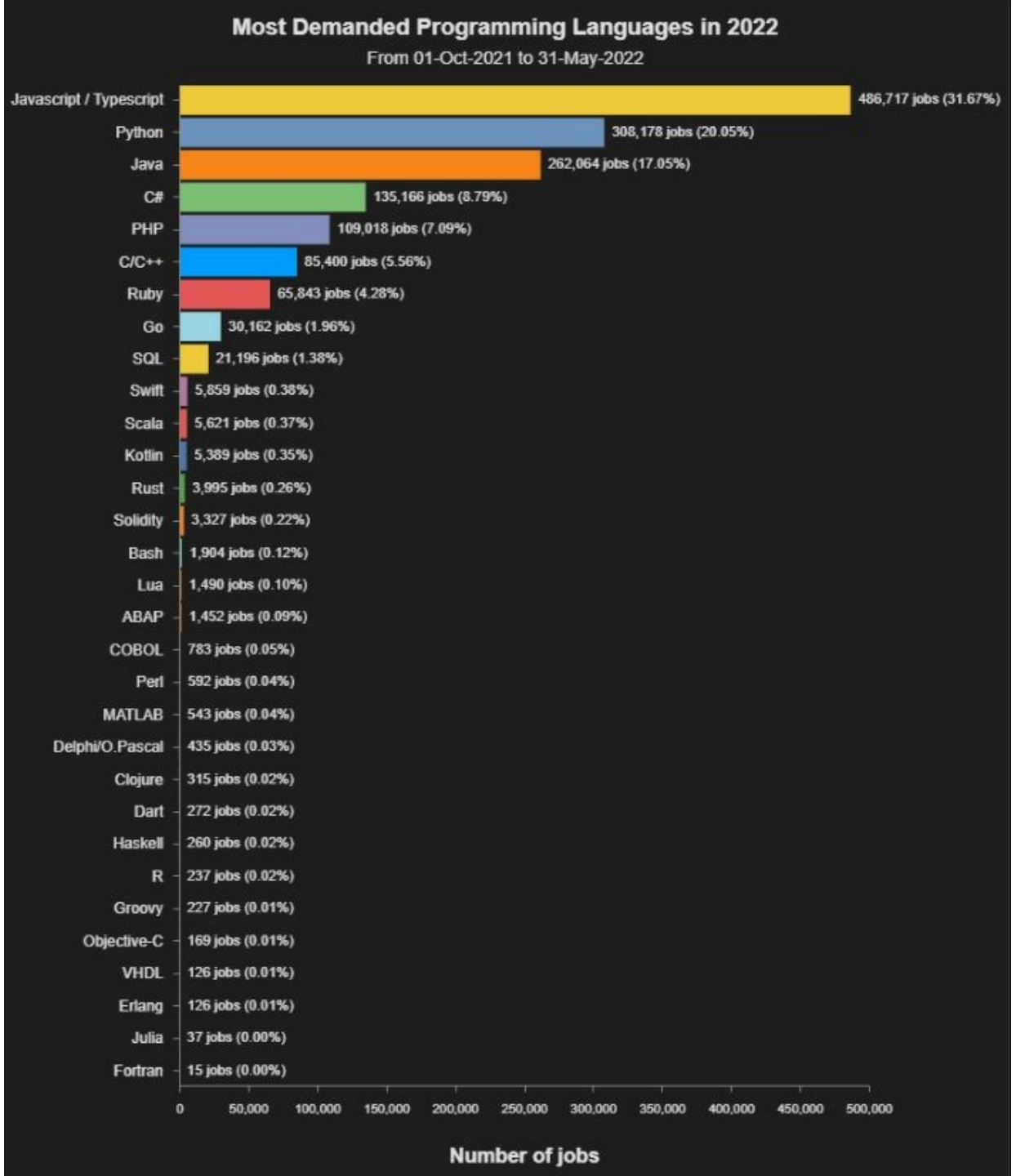
B语言

C语言



Java 作为静态面向对象编程语言（ OOP ）的代表，极好地实现了面向对象理论，使得能以优雅的 OO 方式进行复杂的编程





Software 1.0 ：【传统编程】

特点：程序员用代码一步一步写清楚“规则”。

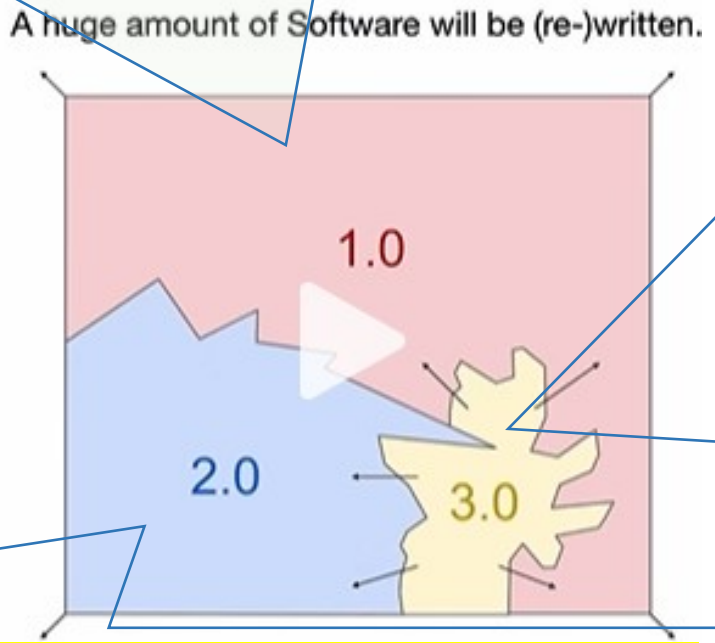
方式：人类先思考逻辑，再把规则写成 if-else 、 for 循环、函数调用。

例子：写一个排序程序时，程序员明确写下比较和交换的步骤。

本质：规则是人写的，计算机只执行。



- 各有优缺点
- 在 3 种范式中流畅转换



Software 3.0 ：【大模型编程】（知识在大模型里，大模型编程，靠 Prompt 和微调）

特点：程序员不需要再自己准备专门的数据和训练，而是直接利用预训练的大模型（如 GPT 、 LLaMA ）。

方式：通过提示词（ prompt ）、指令调优（ fine-tune ）、上下文数据，让大模型完成任务。

例子：想写一个文章摘要工具，不需要收集几百万篇文章再训练模型，而是直接调用 GPT ，并写提示词：“请总结以下文章内容”。

本质：规则和知识都在大模型中，人主要做“提示工程”和“轻量适配”。

Software 2.0 ：【机器学习编程】（规则由机器学，机器学习，靠数据训练）

特点：程序员不再手写规则，而是定义一个模型架构和训练目标。

方式：把数据喂给神经网络，靠优化算法“学出”规则。

例子：图像识别，程序员不会写“眼睛 = 两个黑点，鼻子 = 中间三角形”，而是给模型成千上万张照片，它自动学会“这是人脸”。

本质：规则是机器学的，人写的是“框架”和“目标函数”。

Software 2.0 ：【机器学习编程】（规则由机器学，机器学习，靠数据训练）

特点：程序员不再手写规则，而是定义一个模型架构和训练目标。

方式：把数据喂给神经网络，靠优化算法“学出”规则。

例子：图像识别，程序员不会写“眼睛 = 两个黑点，鼻子 = 中间三角形”，而是给模型成千上万张照片，它自动学会“这是人脸”。

本质：规则是机器学的，人写的是“框架”和“目标函数”。

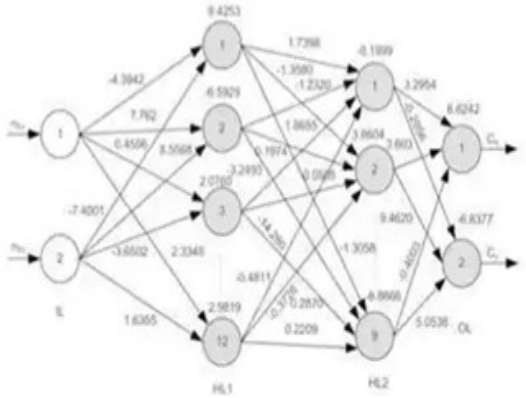
在 Software 1.0 中，人工编写的源代码（例如一些 .cpp 文件）被编译成可执行有用工作的二进制文

1

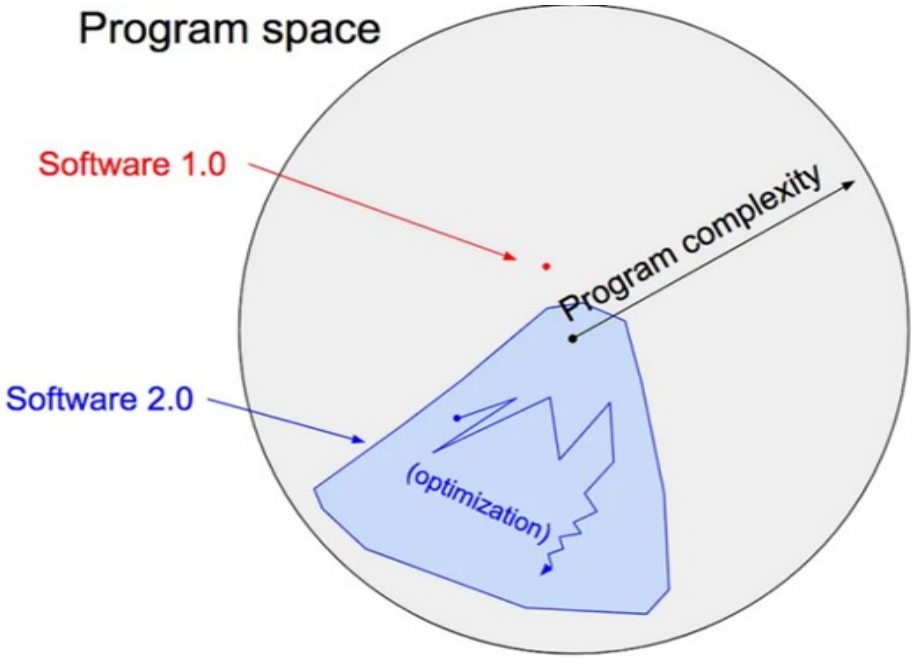
Software 1.0 = code



Software 2.0 = weights



1.0 程序员维护和迭代周围的训练代码基础设施、分析、可视化和标注接口



2.0 程序员（数据标注者）编辑和增加数据集

在 Software 2.0 中，源代码通常包括 1) 定义理想行为的数据集和 2) 给出代码粗略骨架的神经网络架构，但有许多细节（权重）需要填写。训练神经网络的过程将数据集编译成二进制文件——最终的神经网络

Software 1.0

computer code

programs

computer



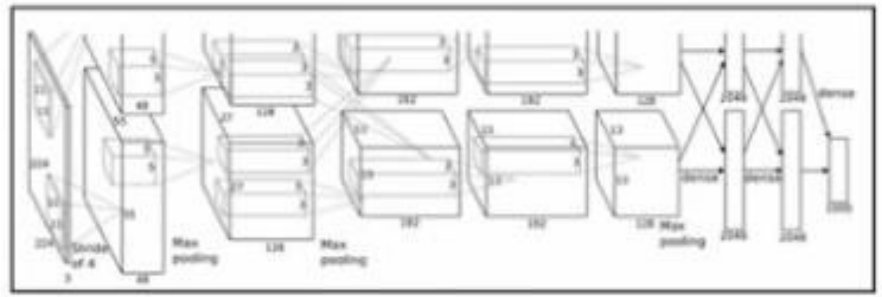
became programmable in ~1940s

Software 2.0

weights

programs

neural net



fixed function neural net

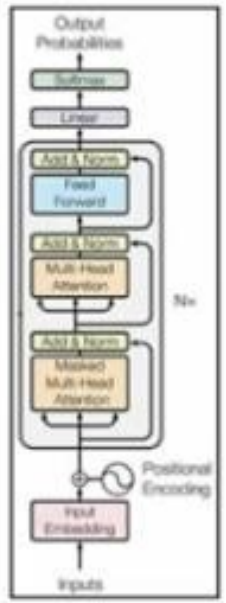
e.g. AlexNet: for image recognition (~2012)

Software 3.0

prompts

programs

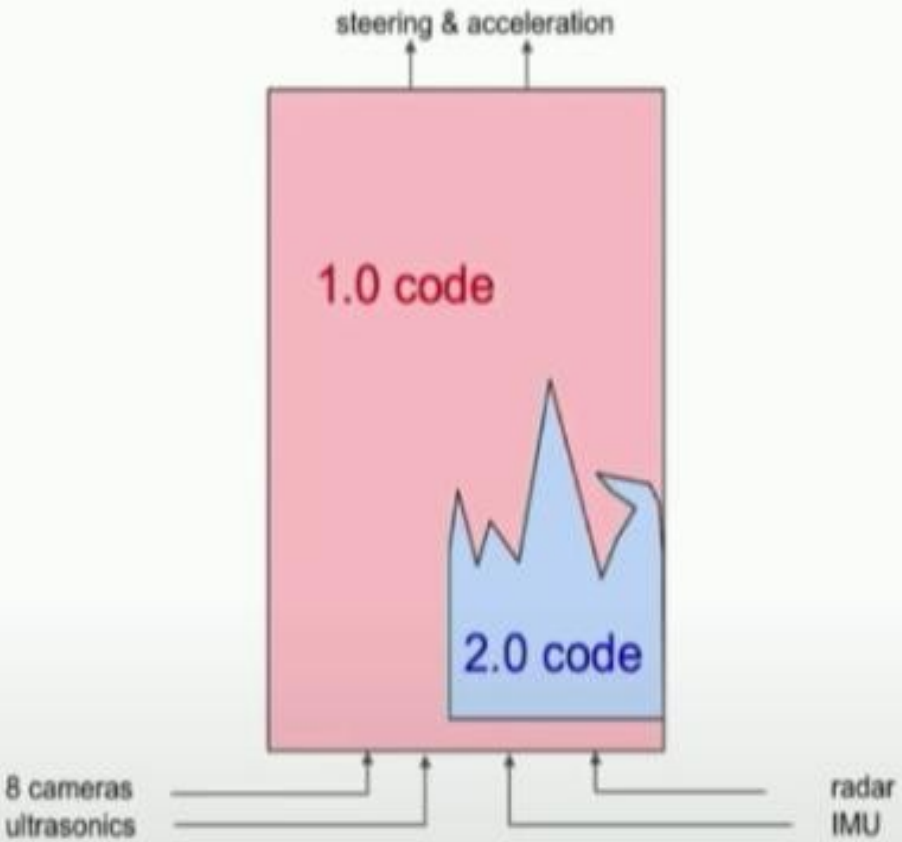
LLM

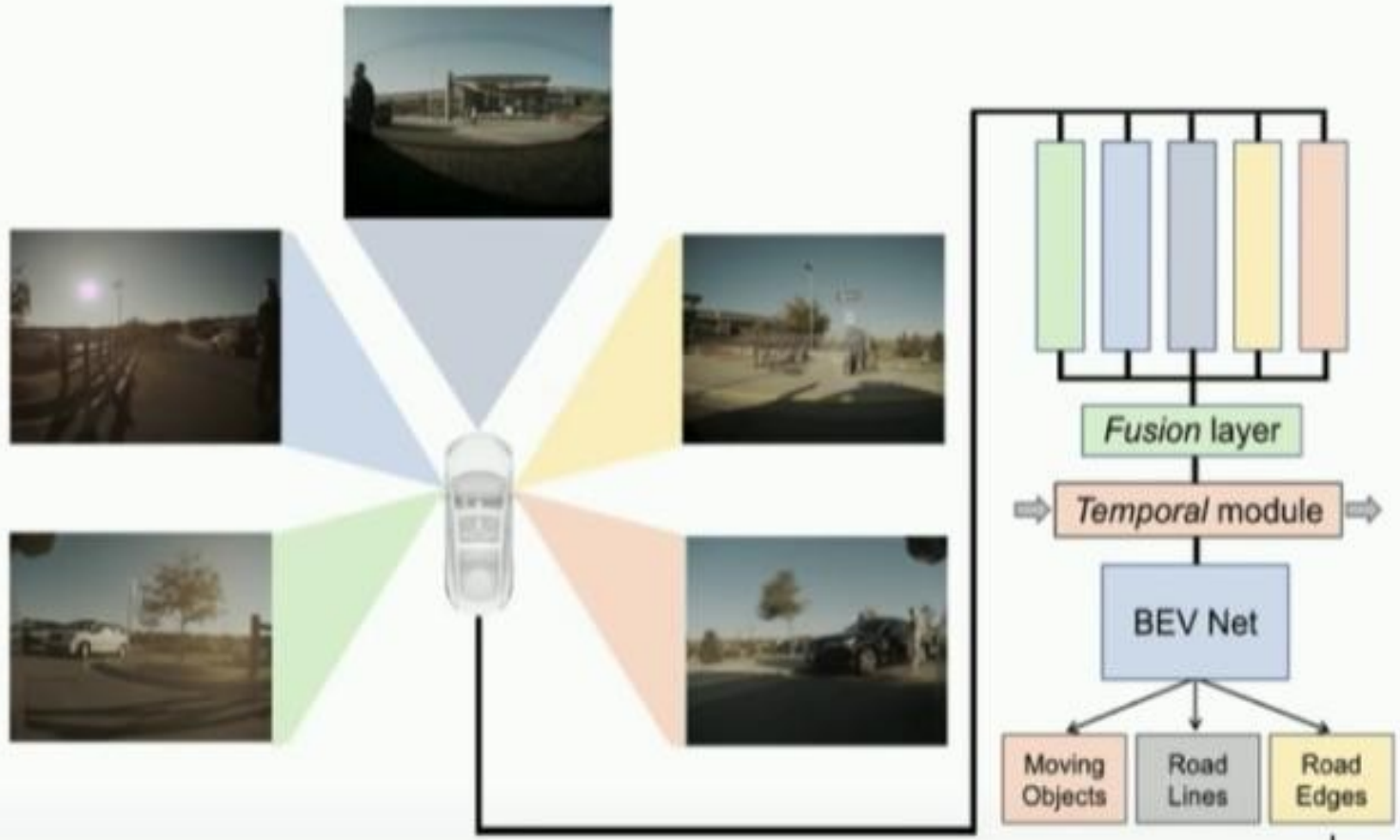


~2019

LLM = programmable neural net!

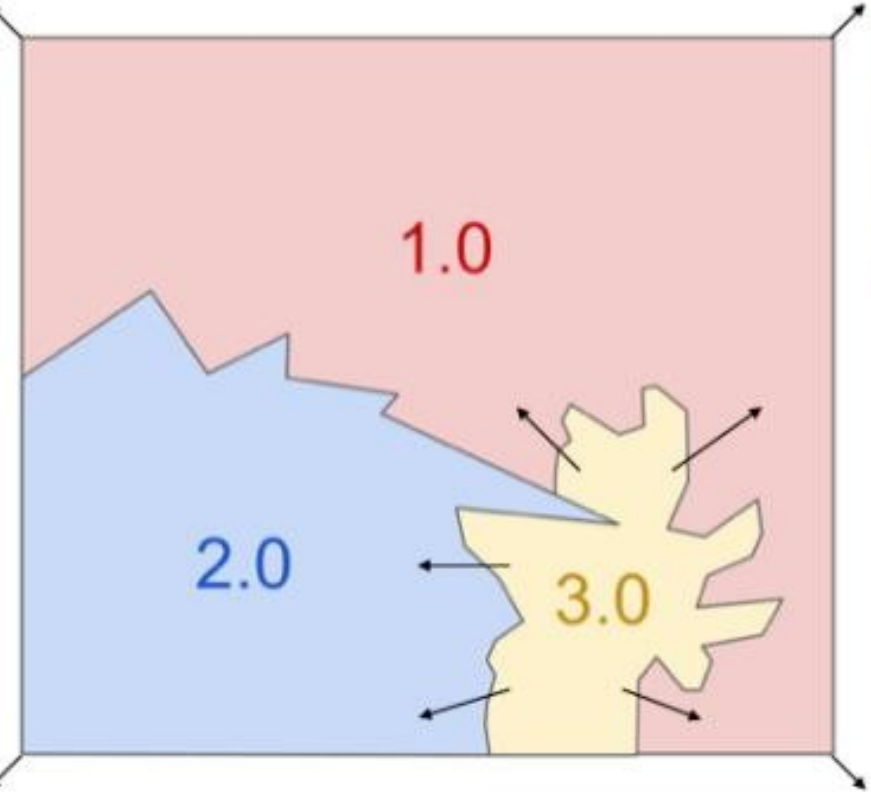
Software is eating the world
Software 2.0 eating Software 1.0





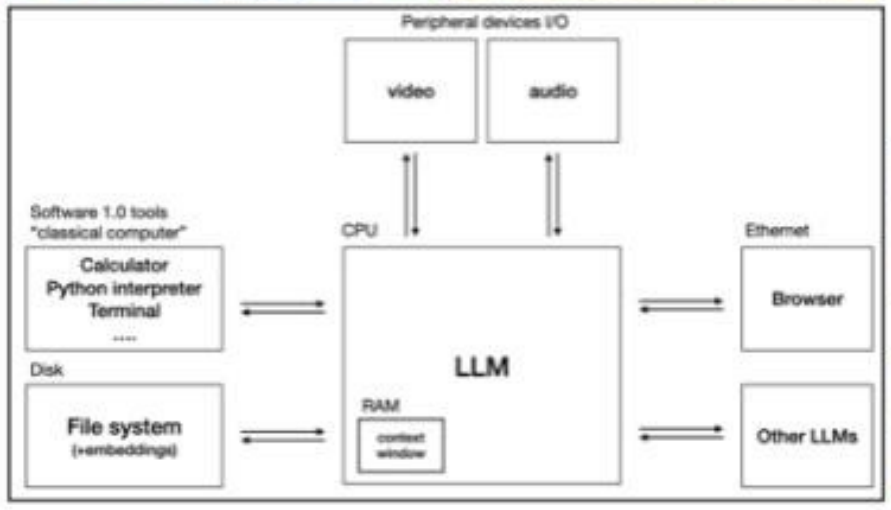
Bird's eye view predictions:















Partial autonomy LLM apps:

- Package context
- Orchestrate LLM calls
- Custom GUI
- Autonomy slider

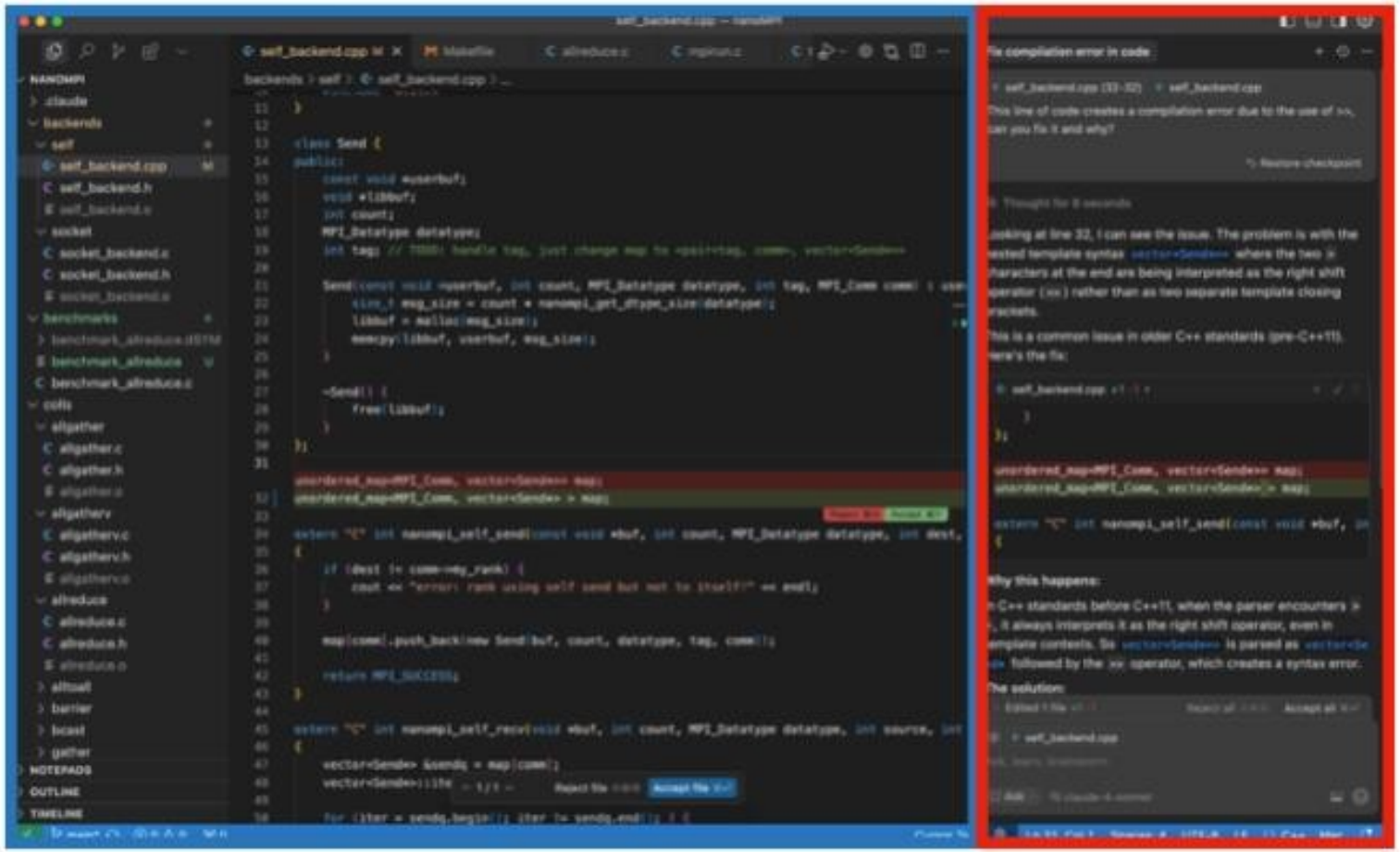


Build for agents 🤖

Example: Anatomy of Cursor

Traditional interface

LLM integration



1. Package state into a context window before calling LLM.

2. Orchestrate and call multiple models (e.g. embedding models, chat models, diff apply models, ...)

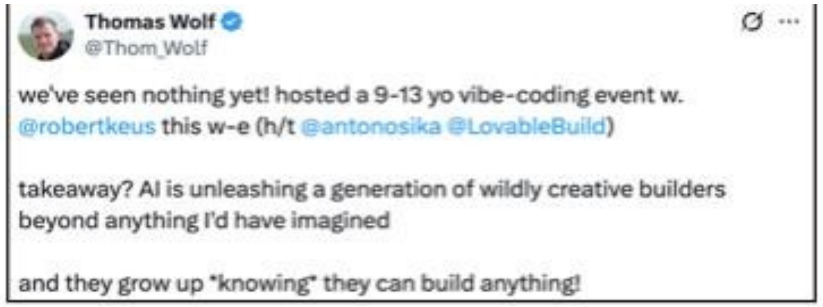
3. Application-specific GUI

4. Autonomy slider: Tab → Cmd+K → Cmd+L → Cmd+I (agent mode)

autonomy slider

Vibe Coding 通过自然语言提示与 **AI** 协作生成可运行代码的新型软件开发范式

(《韦氏英语惯用法词典》2025)



案例
https://www.zhihu.com/question/15685495624/answer/1934918085397153639



为什么 Karpathy 说软件将会被重写

排序	理由	解释	相关性
1	**编程范式改变**	从手写规则（1.0）→ 机器学习（2.0）→ 大模型驱动（3.0），逻辑表达方式彻底转变，旧的软件模式不再适用。	★★★★
2	**效率与成本优势**	大模型能以更低成本、更快速度实现复杂功能，企业有强烈动力用新方式重构旧系统。	★★★★
3	**功能边界扩展**	大模型带来全新能力（自然语言接口、自动化、智能化），旧系统难以满足新需求，需要重写。	★★★
4	**技术栈更新迭代**	新的软件栈（Prompt、模型调用、微调）与旧逻辑不兼容，推动系统迁移与重构。	★★

语言“家族”特性

Java 语言平台概述

◆ Java 技术体系平台

Java SE(Java Standard Edition) 标准版
支持面向**桌面级应用**（如 Windows 下的应用程序）的 Java 平台，提供了完整的 Java 核心 API ，此版本以前称为 **J2SE**
Java EE(Java Enterprise Edition) 企业版
是为开发企业环境下的应用程序提供的一套解决方案。该技术体系中包含的技术如 :Servlet 、 Jsp 等，主要针对于 **Web 应用程序开发**。版本以前称为 **J2EE**
Java ME(Java Micro Edition) 小型版
支持 Java 程序运行在**移动终端**（手机、 PDA ）上的平台，对 Java API 有所精简，并加入了针对移动终端的支持，此版本以前称为 **J2ME**
Java Card
支持一些 **Java 小程序**（ Applets ）运行在小内存设备（如智能卡）上的平台

Java 在各领域中的应用

语言“家族”特性
“语用”的细分

- 从 Java 的应用领域来分，Java 语言的应用方向主要表现在以下几个方面：
  - **企业级 Web 应用**：主要指复杂的大企业的软件系统、各种类型的网站。Java 的安全机制以及它的**跨平台的优势，使它在分布式系统领域开发中有广泛应用**。应用领域包括金融、电信、交通、电子商务等。
  - **Android 平台应用**： Android 应用程序使用 Java 语言编写。 Android 开发水平的高低很大程度上取决于 Java 语言核心能力是否扎实。
  - 移动领域应用，主要表现在消费和嵌入式领域，是指在各种小型设备上的应用，包括手机、 PDA 、机顶盒、汽车通信设备等。

移动开发 VS 企业级开发

语言“家族”特性
“语用”的细分











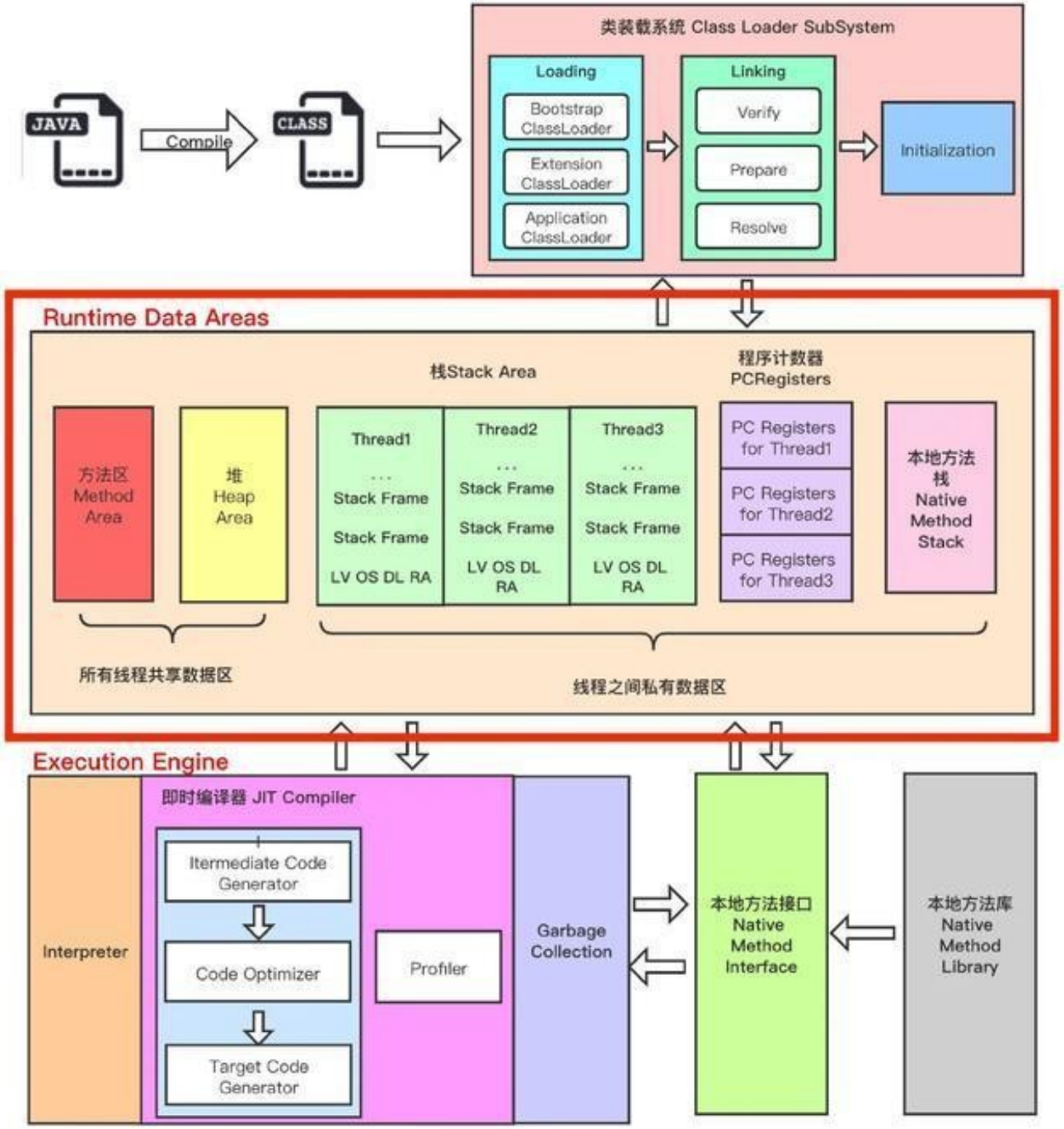
Java 语言运行机制及运行过程

- Java 两种核心机制

  - Java 虚拟机（ Java Virtual Machine ）
  - 垃圾收集机制（ Garbage Collection ）

2 个机制

JAVA 执行过程



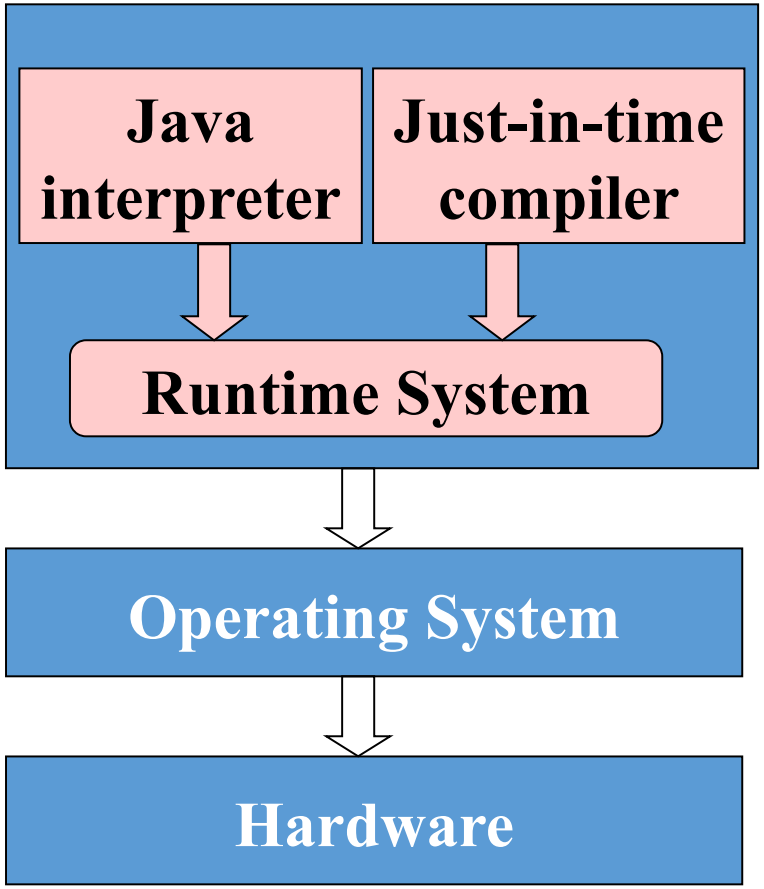
第 1 个机制— JVM 代码与运行环境分离接口的概念

第 1 个机制— JVM
代码与运行环境分离
接口的概念

◆ 核心机制— Java 虚拟机

- **JVM 是一个虚拟的计算机，具有指令集并使用不同的存储区域。负责执行指令，管理数据、内存、寄存器。**
- 对于不同的平台，有不同的虚拟机。
- Java 虚拟机机制屏蔽了底层运行平台的差别，实现了“一次编译，到处运行”（ Write Once ， Run any Where ）。

• Java 虚拟机



不同的操作系统有不同的虚拟机 .

它类似一个小巧而高效的 CPU.

byte-code 代码是与平台无关的

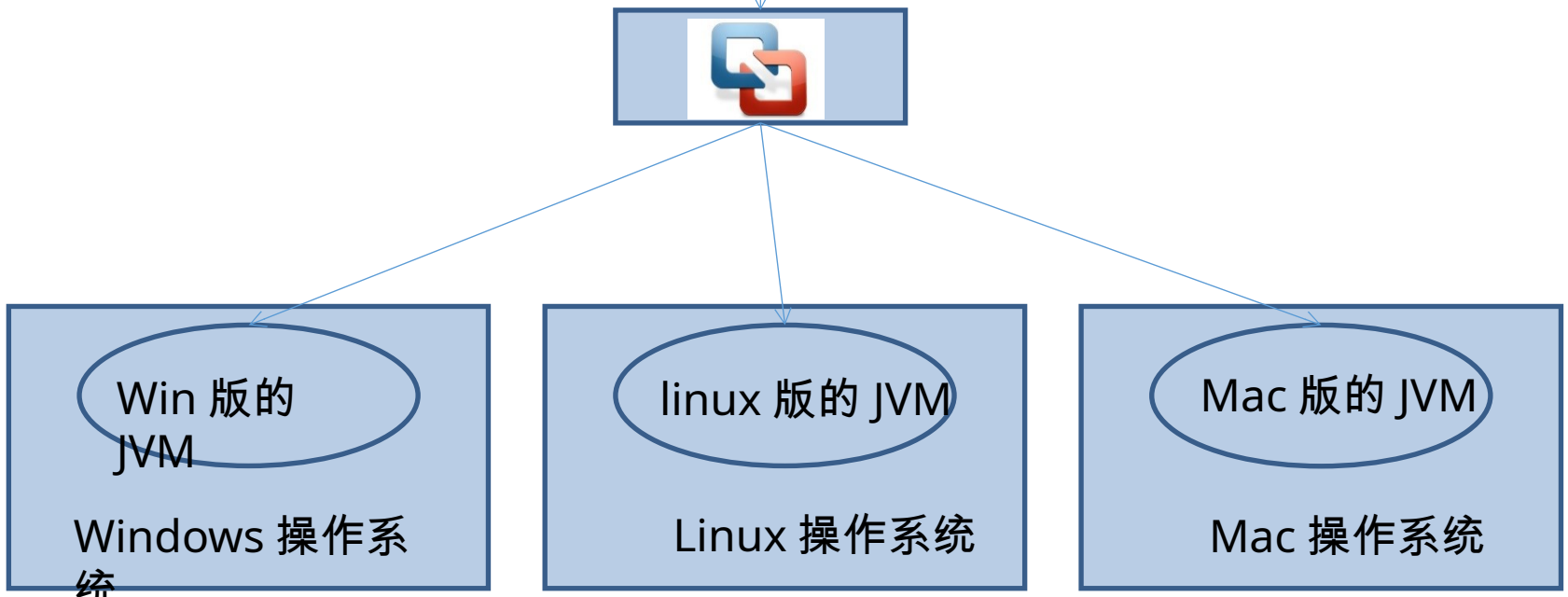
是虚拟机的机器指令 .

Java 字节代码运行的两种方式 :

• interpreter(解释方式)

• Just-in-time(即时编译): 代码生成器将字节代码转换成本机的机器代码，就可以以较高速度执行 .

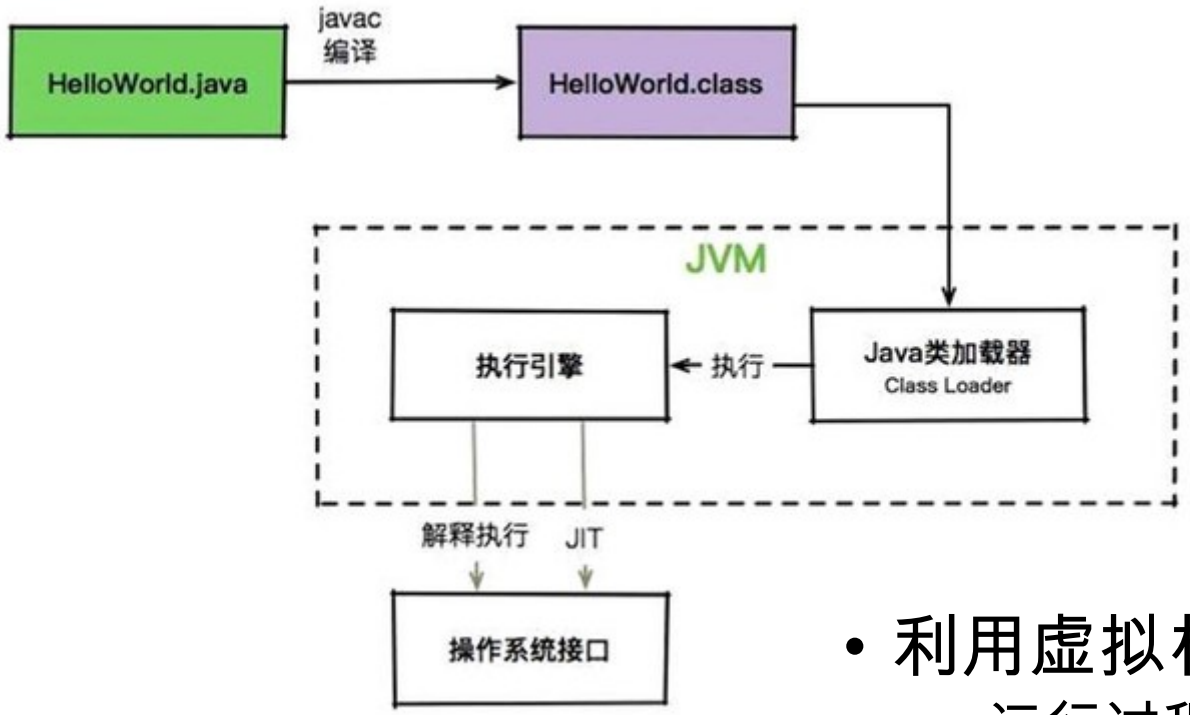
```java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```



第 1 个机制— JVM 代码与运行环境分离接口的概念

因为有了 JVM ，同一个 Java 程序在三个不同的操作系统中都可以执行。这样就实现了 Java 程序的跨平台性。

- 利用编辑器编写 Java 源程序
  - 源文件名：主类名 .java
- 利用编译器将源程序编译成字节码
  - 字节码文件名：源文件名 .class



第 1 个机制—JVM 代码执行过程

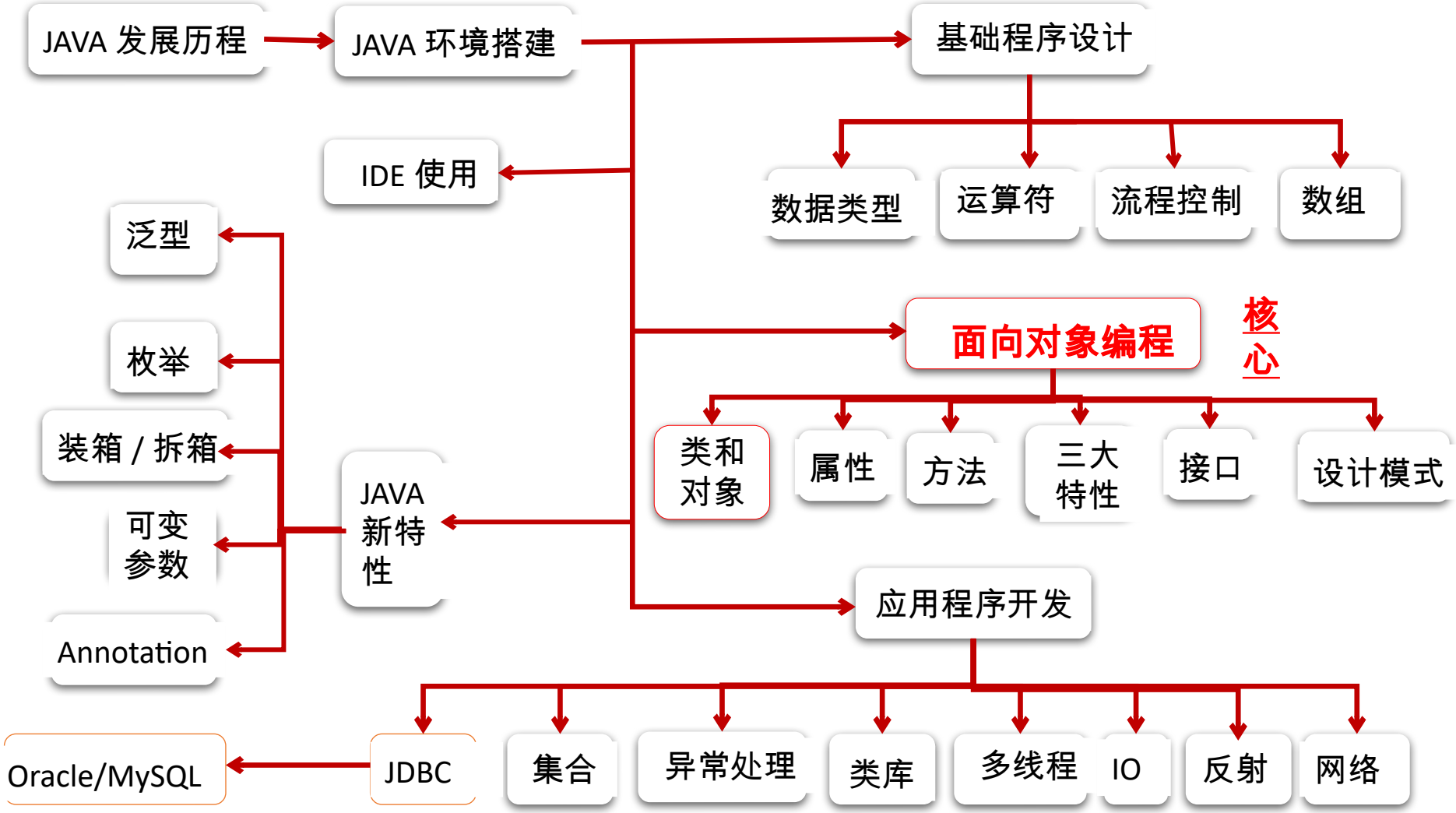
- 利用虚拟机（解释器）运行
  - 运行过程：载入、代码校验、解释执行

◆ 核心机制—垃圾回收

- 不再使用的内存空间应回收—— 垃圾回收，垃圾回收可以有效的防止内存泄露，有效的使用空闲的内存。
  - Java 中的对象不再有“作用域”的概念，只有对象的引用才有“作用域”。
  - 使得 Java 程序员在编写程序的时候不再需要考虑内存管理：垃圾回收机制提供一种系统级线程跟踪存储空间的分配情况，并在 JVM 空闲时，检查并释放那些可被释放的存储空间。
- 垃圾回收在 Java 程序运行过程中自动进行，程序员无法精确控制和干预。

Java 基础知识图解

课程路线图



- 编程语言核心结构：

➢变量、基本语法、分支、循环、数组、……

- java 面向对象的核心逻辑：

➢ OOP 、封装、继承、多态、接口……

- 开发 java se 高级应用程序：

➢异常、集合、 IO 、多线程、反射机制、网络编程……

学习建议

- 锻炼“双核”处理，边听讲思考，边做“笔记”
- 纸上得来终觉浅，绝知此事要躬行！
  - 不要完全依赖于书和视频
- 建立行之有效的学习方法
  - 学习编程的捷径 -- **敲代码**
  - 学习编程的规范 -- **加适量注释**

一些相关研究工作
1）注释生成、更新
2）Code Summary

第 1 讲 Java 程序设计简介

1.1 Java 发展历程

1.2 Java 环境搭建

1.3 Java 程序基本结构

1.4 Eclipse 集成开发环境

Java 环境配置

- Java 是一门编译型语言、静态语言，使用 JDK 将源代码编译为字节码（ byte code ）。
- Java 开发环境（ JDK ）分为 Java SE 、 Java EE 、 Java ME 三个版本，**Java SE 的最新版本为 JDK-22（ 2024 年）**。

Java 语言的环境搭建

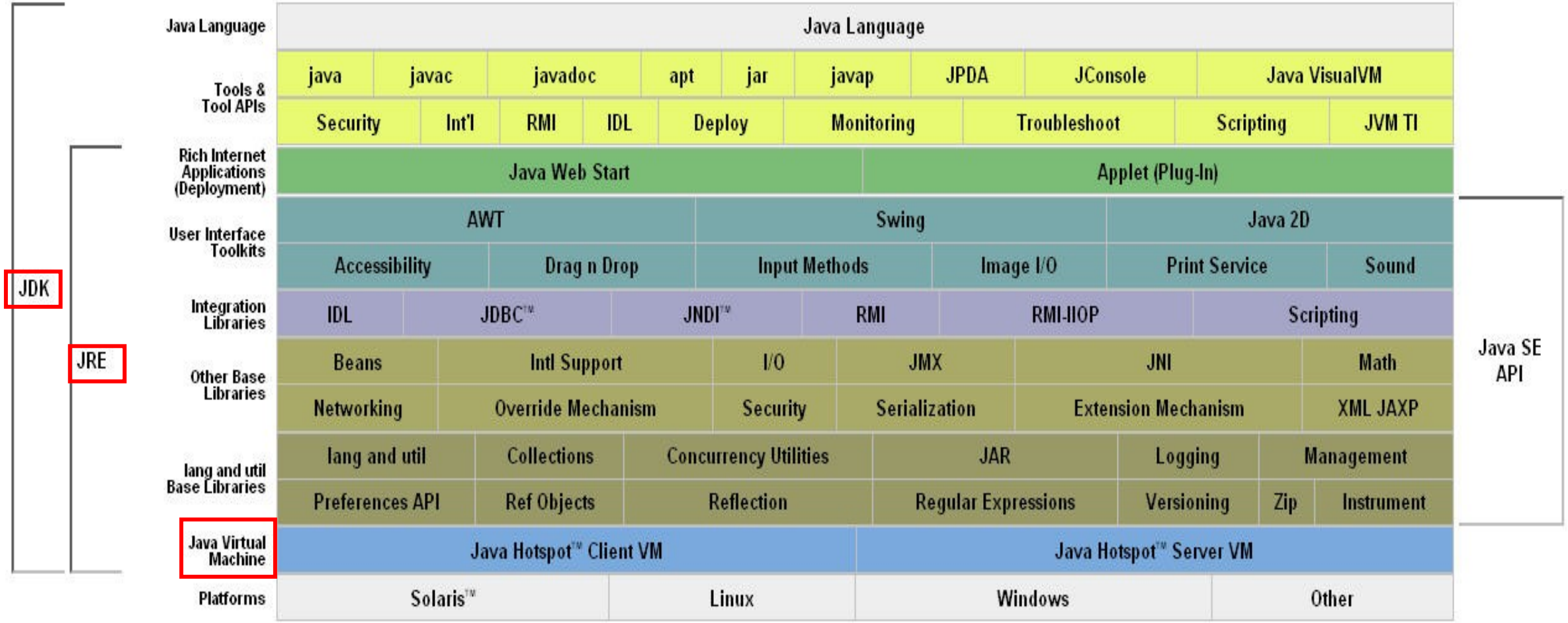
- 明确什么是 JDK, JRE
- 下载 JDK
- 安装 JDK
- 配置环境变量
  - **path** ： windows 系统执行命令时要搜寻的路径。
  - **classpath** ： java 在编译和运行时要找的 class 所在的路径。
- 验证是否成功： javac java
- 选择合适的文本编辑器或 IDE 开发

配置环境变量

◆ 什么是 JDK ， JRE

JDK(Java Development Kit　Java 开发工具包)
JDK 是提供给 Java 开发人员使用的，其中包含了 java 的开发工具，也包括了 JRE 。所以安装了 JDK ，就不用在单独安装 JRE 了。 ➢ 其中的开发工具：编译工具 (javac.exe)　打包工具 (jar.exe) 等
JRE(Java Runtime Environment　Java 运行环境)
包括 Java 虚拟机 (JVM Java Virtual Machine) 和 Java 程序所需的核心类库等，如果想要**运行**一个开发好的 Java 程序，计算机中只需要安装 JRE 即可。 **简单而言，使用 JDK 的开发工具完成的 java 程序，交给 JRE 去运行。**

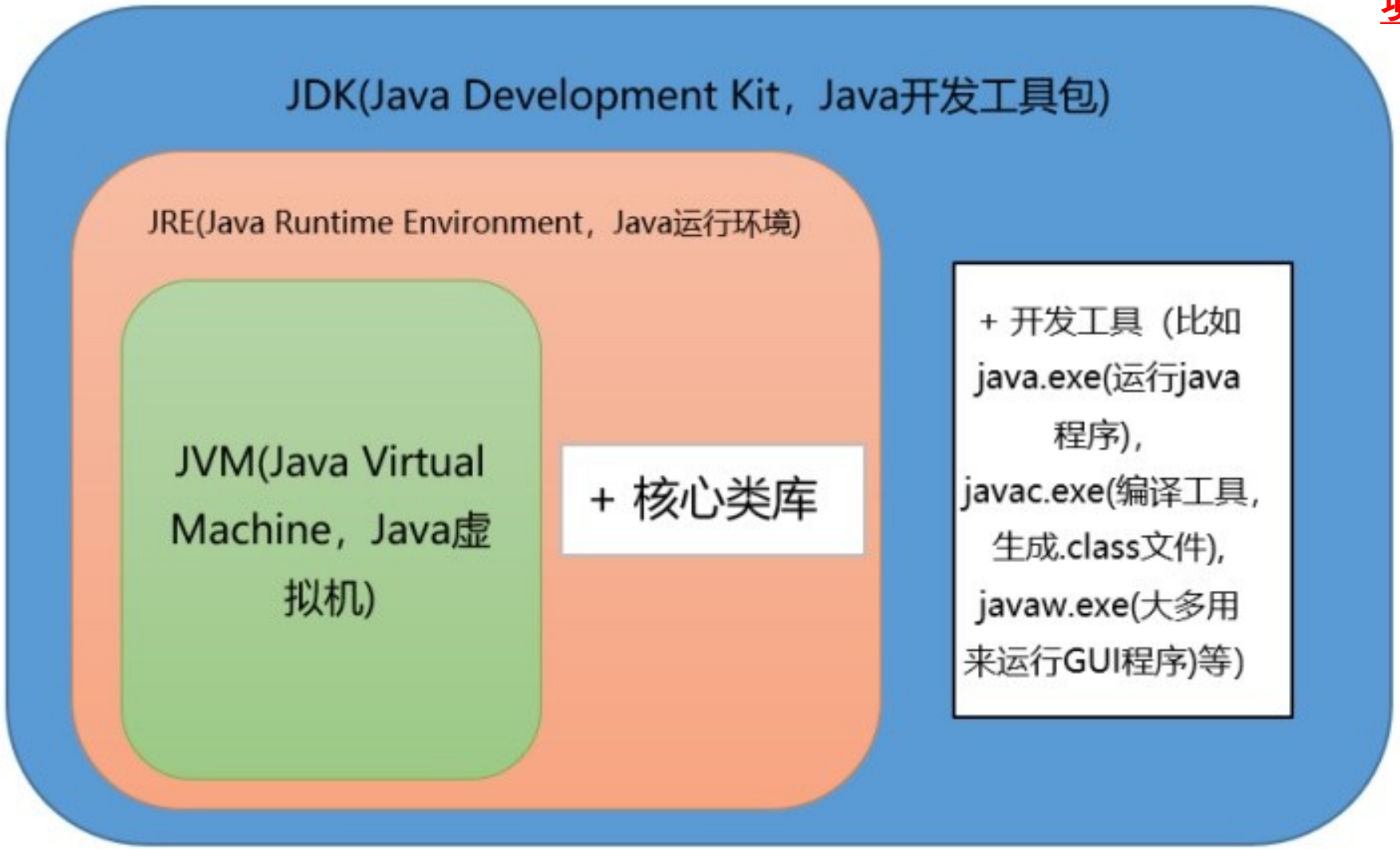
JVM 虚拟机、 JRE 运行环境、 JDK 开发工具、之间的关系



JVM⊂JRE⊂JDK

JVM 、 JRE 、 JDK

JAVA 编程环境


JDK(Java Development Kit，Java开发工具包)
JRE(Java Runtime Environment，Java运行环境)
JVM(Java Virtual Machine，Java虚拟机)
+ 核心类库
+ 开发工具（比如java.exe(运行java程序)，javac.exe(编译工具，生成.class文件)，javaw.exe(大多用来运行GUI程序)等）

◆ 下载、安装 JDK

下载 JDK

- 官方网址：
  - www.oracle.com
  - java.sun.com
- 安装 JDK
  - 傻瓜式安装，下一步即可。
  - 建议：安装路径不要有中文或者特殊符号如空格等。
  - 当提示安装 JRE 时，可以选择不安装。

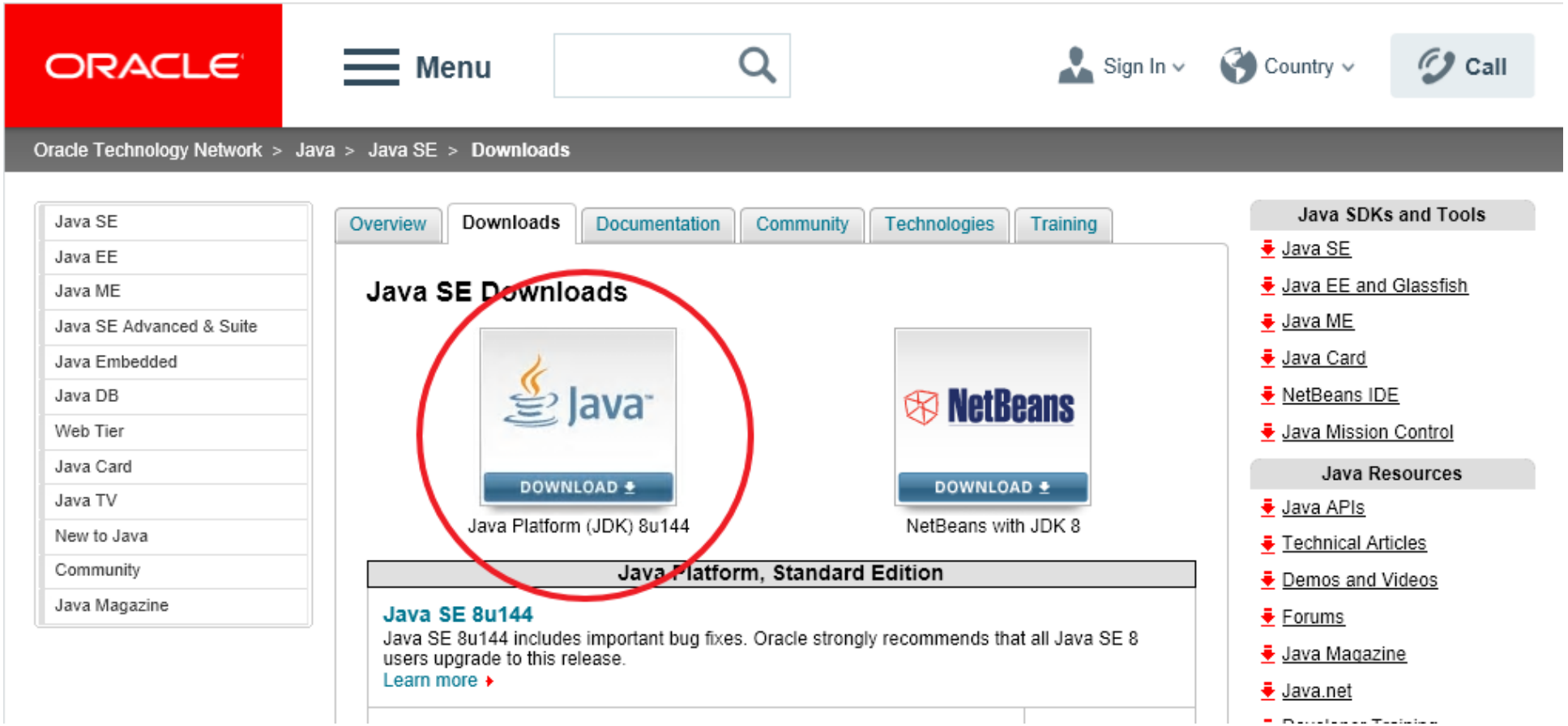
下载 JDK

下载
JDK


Bai du 百度
JDK
百度一下
网页 新闻 贴吧 知道 音乐 图片 视频 地图 文库 更多»
百度为您找到相关结果约34,200,000个
搜索工具
您可以仅查看：英文结果
Java SE - Downloads | Oracle Technology Network | Oracle
查看此网页的中文翻译，请点击 翻译此页
Java SE downloads including: Java Development Kit (JDK), Server Java Runtime Environme
nt (Server JRE), and Java Runtime Environment (JRE).
www.oracle.com/technet... - 百度快照

下载 JDK

下载 JDK


ORACLE
Menu
Sign In
Country
Call
Oracle Technology Network > Java > Java SE > Downloads
Java SE
Java EE
Java ME
Java SE Advanced & Suite
Java Embedded
Java DB
Web Tier
Java Card
Java TV
New to Java
Community
Java Magazine
Overview
Downloads
Documentation
Community
Technologies
Training
Java SE Downloads
Java
DOWNLOAD
Java Platform (JDK) 8u144
NetBeans
DOWNLOAD
NetBeans with JDK 8
Java Platform, Standard Edition
Java SE 8u144
Java SE 8u144 includes important bug fixes. Oracle strongly recommends that all Java SE 8 users upgrade to this release.
Learn more
Java SDKs and Tools
Java SE
Java EE and Glassfish
Java ME
Java Card
NetBeans IDE
Java Mission Control
Java Resources
Java APIs
Technical Articles
Demos and Videos
Forums
Java Magazine
Java.net

下载 JDK

下载 JDK

- Java SE
- Java EE
- Java ME
- Java SE Advanced & Suite
- Java Embedded
- Java DB
- Web Tier
- Java Card
- Java TV
- New to Java
- Community
- Java Magazine

Overview | Downloads | Documentation | Community | Technologies | Training

Java SE Development Kit 8 Downloads

Thank you for downloading this release of the Java™ Platform, Standard Edition Development Kit (JDK™). The JDK is a development environment for building applications, applets, and components using the Java programming language.

The JDK includes tools useful for developing and testing programs written in the Java programming language and running on the Java platform.

Java SE Development Kit 8u144

You must accept the Oracle Binary Code License Agreement for Java SE to download this software.

○ Accept License Agreement ○ Decline License Agreement

Product / File Description	File Size	Download
Linux ARM 32 Hard Float ABI	77.89 MB	jdk-8u144-linux-arm32-vfp-hflt.tar.gz
Linux ARM 64 Hard Float ABI	74.83 MB	jdk-8u144-linux-arm64-vfp-hflt.tar.gz
Linux x86	164.65 MB	jdk-8u144-linux-i586.rpm
Linux x86	179.44 MB	jdk-8u144-linux-i586.tar.gz
Linux x64	162.1 MB	jdk-8u144-linux-x64.rpm
Linux x64	176.92 MB	jdk-8u144-linux-x64.tar.gz
Mac OS X	226.6 MB	jdk-8u144-macosx-x64.dmg
Solaris SPARC 64-bit	139.87 MB	jdk-8u144-solaris-sparcv9.tar.Z
Solaris SPARC 64-bit	99.18 MB	jdk-8u144-solaris-sparcv9.tar.gz
Solaris x64	140.51 MB	jdk-8u144-solaris-x64.tar.Z
Solaris x64	96.99 MB	jdk-8u144-solaris-x64.tar.gz
Windows x86	190.94 MB	jdk-8u144-windows-i586.exe
Windows x64	197.78 MB	jdk-8u144-windows-x64.exe

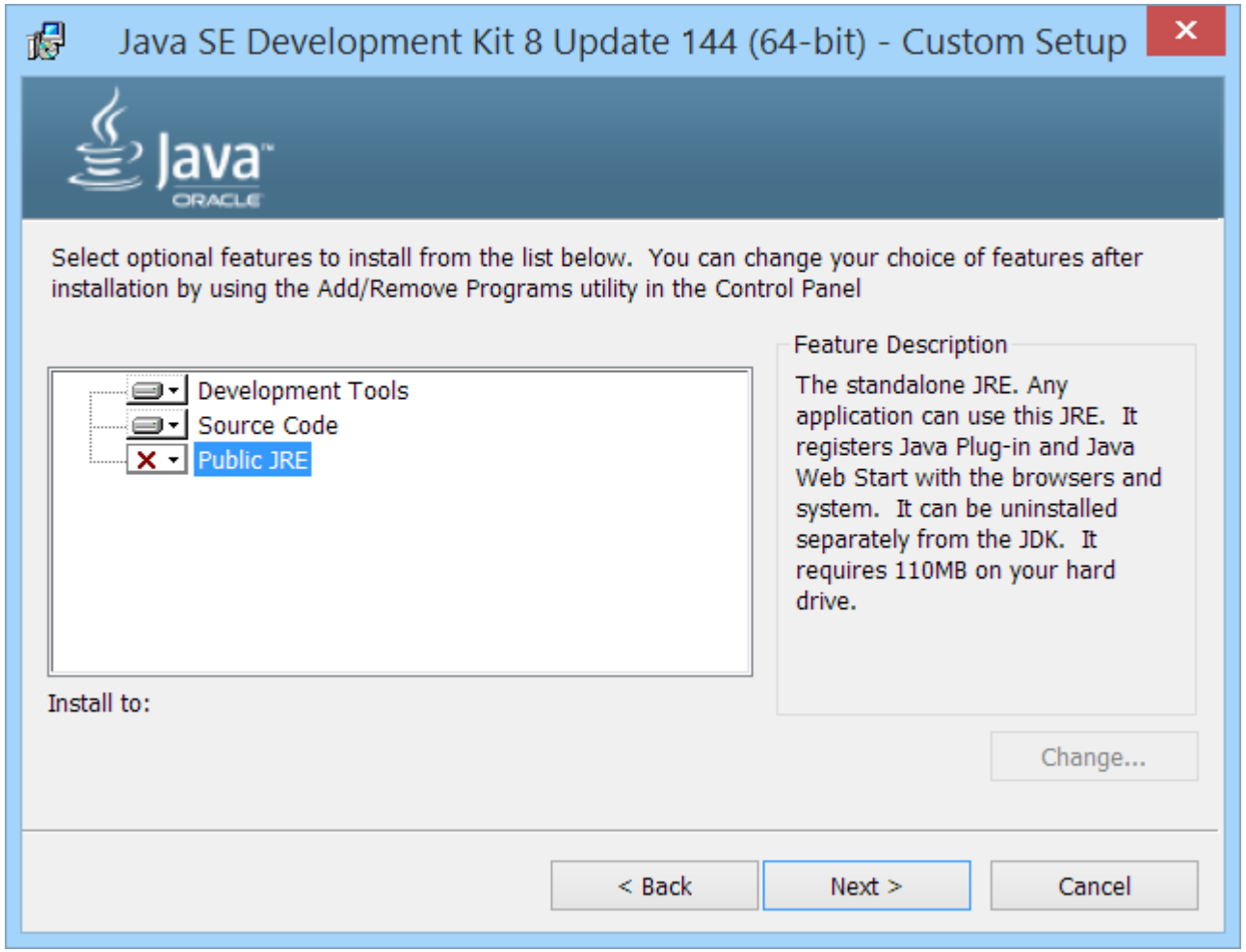
Java SDKs and Tools

- Java SE
- Java EE and Glassfish
- Java ME
- Java Card
- NetBeans IDE
- Java Mission Control

Java Resources

- Java APIs
- Technical Articles
- Demos and Videos
- Forums
- Java Magazine
- Java.net
- Developer Training
- Tutorials
- Java.com

安装 JDK



安装 JDK

当提示安装 JRE 时，可以选择不安装

安装JDK

- JDK8 中主要的目录和文件有：

➢ bin 目录： Java 工具的可执行文件，包括 : java 、 javac 、 javap 、 keytool （ 密钥管理工具 ）、 javadoc （ Java 文档工具 ）等。

➢ jre 目录： Java 运行时环境。

➢ include 目录：包含一些 C 语言开发时用到的 .h 头文件。比如 jni.h 是开发 jni 程序时必须引用的头文件。

➢ lib 目录： Java 类库，我们经常看到的 dt.jar 和 tools.jar 就在这个目录下。

➢ db 目录： Java 实现的数据库。

➢ src.zip 文件： Java 类库源码，包括了 rt.jar 库中的关键部分；除了 Java 类库，还包含了启动器（ launcher ）的源码（ C 语言实现 ）。

➢ COPYRIGHT 文件：当前版本的 JRE 的版权信息文件。

◆ 配置环境变量 path

- “ Win+R ”“ cmd ”，在 dos 命令行中敲入 javac ，出现错误提示：

```
D:\jdk1.7.0_07>javac
'javac' 不是内部或外部命令，也不是可运行的程序
或批处理文件。
```

- 错误原因：**当前执行的程序在当前目录下如果不存在，windows 系统会在系统中已有的一个名为 path 的环境变量指定的目录中查找**。如果仍未找到，会出现以上的错误提示。所以进入到 jdk 安装路径 \bin 目录下，执行 javac ，会看到 javac 参数提示信息。

```
D:\jdk1.7.0_07\bin>javac.exe
用法: javac <options> <source files>
其中, 可能的选项包括:
  -g                         生成所有调试信息
  -g:none                    不生成任何调试信息
```

◆ 配置环境变量 path

每次执行 java 的工具都要进入到 bin 目录下，是非常麻烦的。可不可以在任何目录下都可以执行 java 的工具呢？

- 根据 windows 系统在查找可执行程序的原理，可以将 java 工具所在路径定义到 path 环境变量中，让系统帮我们去找运行执行的程序。
- **配置方法：**
  - 我的电脑 -- 属性 -- 高级系统设置 -- 环境变量
  - 编辑 path 环境变量，在变量值开始处加上 java 工具所在目录，后面用“；”和其他值分隔开即可。
  - 打开 DOS 命令行，任意目录下敲入 javac 。如果出现 javac 的参数信息，配置成功。

注： **具体操作流程，教辅说明**

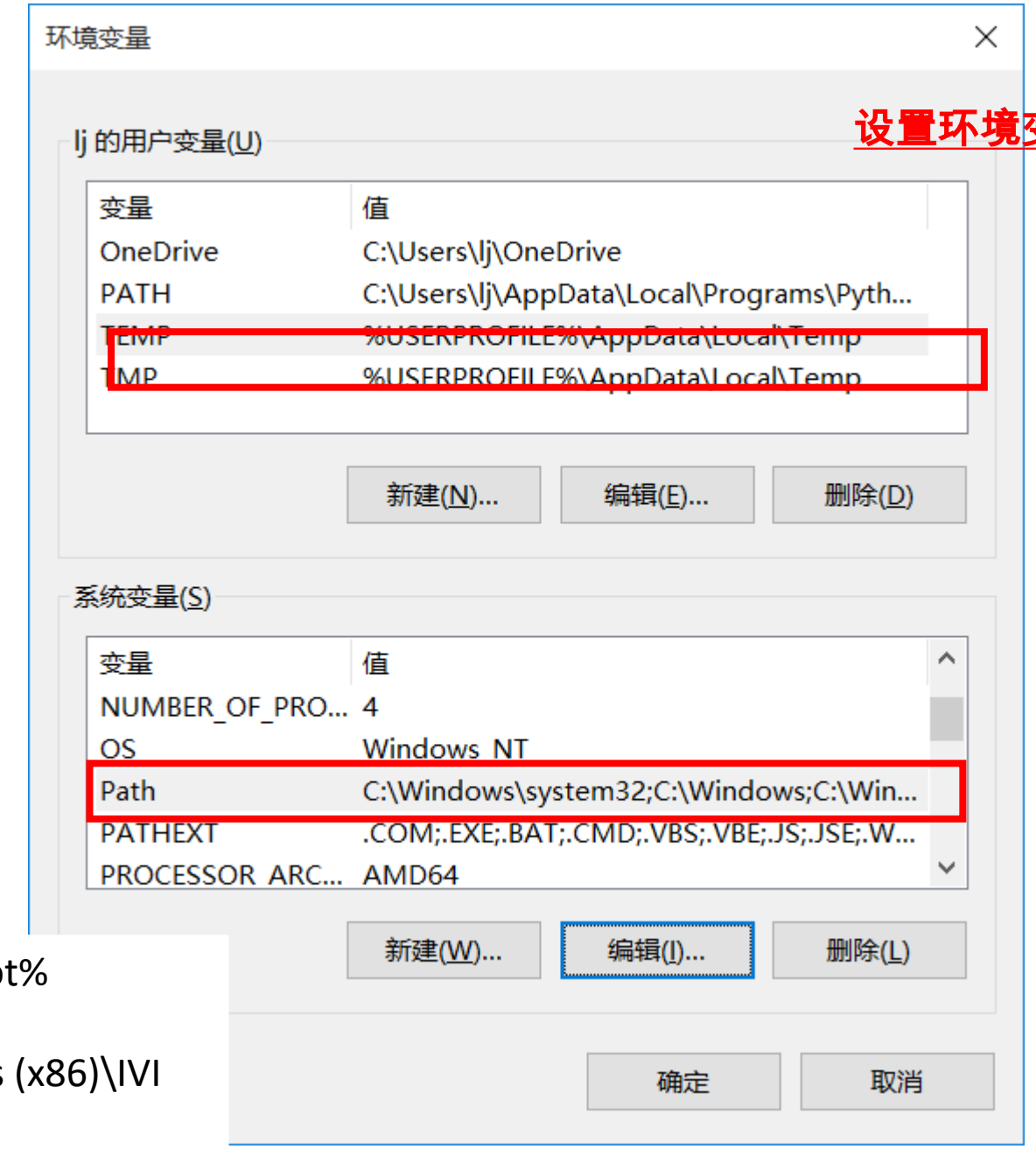
◆ 配置环境变量 path(补充)

- **临时配置方式**：通过 dos 命令中 set 命令完成
  - set ：用于查看本机的所有环境变量的信息。
  - set 变量名 ：查看具体一个环境变量的值。
  - set 变量名 = ：清空一个环境变量的值。
  - set 变量名 = 具体值 ：给指定变量定义具体值。

- 想要在原有环境变量值基础上添加新值呢？
  - 首先，通过 % 变量名 % 操作符获取到原有环境变量的值。
  - 然后加上新值后再定义给该变量名即可
  - 举例：给 path 环境变量加入新值

    set path= 新值 ;%path%
- 注：这种配置方式只在当前 dos 窗口有效。窗口关闭，配置消失。





设置环境变量

%SystemRoot%\system32;%SystemRoot%;%SystemRoot%\System32\Wbem;%SYSTEMROOT%\System32\WindowsPowerShell\v1.0\;C:\Program Files (x86)\IVI Foundation\VISA\WinNT\Bin;**C:\Program Files\Java\jdk1.8.0_144\bin**

第 1 讲 Java 程序设计简介

1.1 Java 发展历程

1.2 Java 环境搭建

1.3 Java 程序基本结构

编程方式

图形 IDE 方式和命令行方式

- **软件开发**

  软件，即一系列按照特定顺序组织的计算机数据和指令的集合。

  有**系统软件**和**应用软件**之分。

- **人机交互方式**

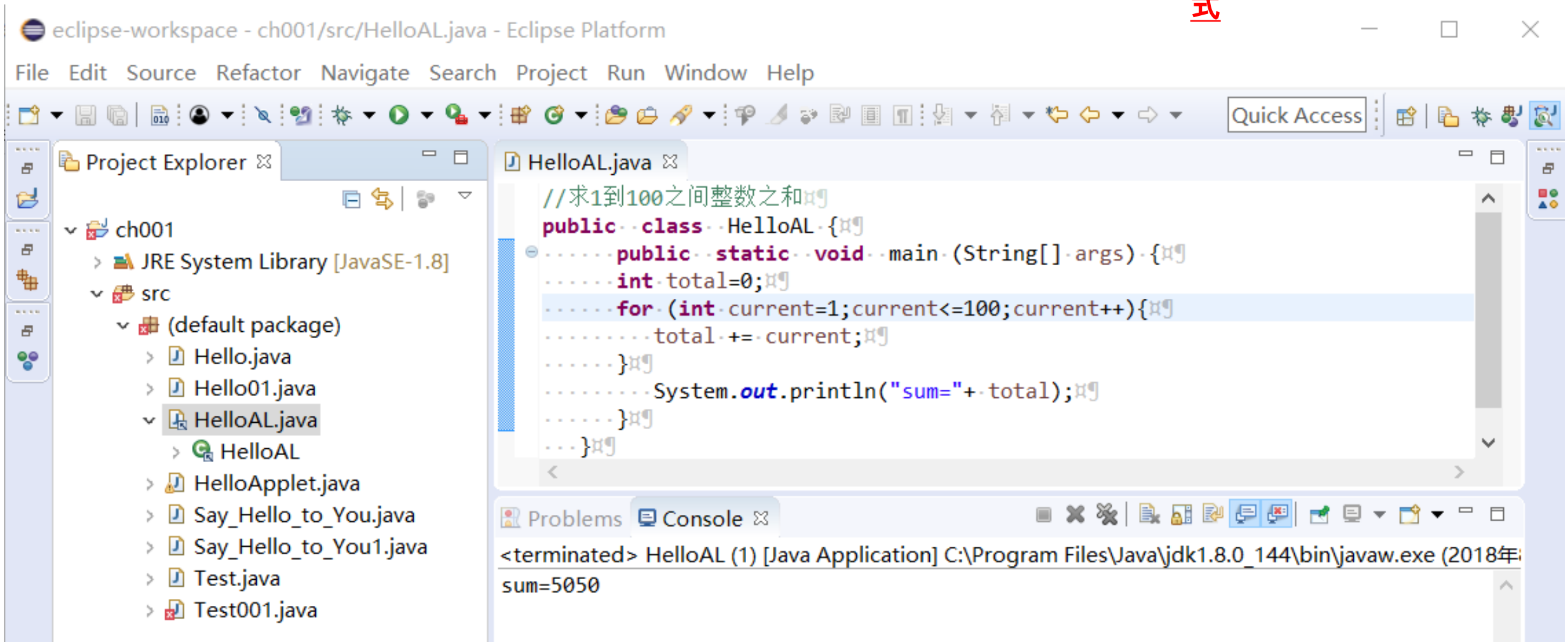
  - 图形化界面 (Graphical User Interface GUI) 这种方式简单直观，使用者易于接受，容易上手操作。
  - 命令行方式 (Command Line Interface CLI) ：需要有一个控制台，输入特定的指令，让计算机完成一些操作。较为麻烦，需要记录住一些命令。

N·Wirth ：“ Algorithms+Data Structures=Programs”

程序 = 算法 + 数据结构

图形化界面方式

图形 IDE 方式



命令行方式

实例 文件 **HelloWorld.java(文件名需与类名一致)**, 代码如下:

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

运行以上实例，输出结果如下:

```
$ javac HelloWorld.java
$ java HelloWorld
Hello World
```

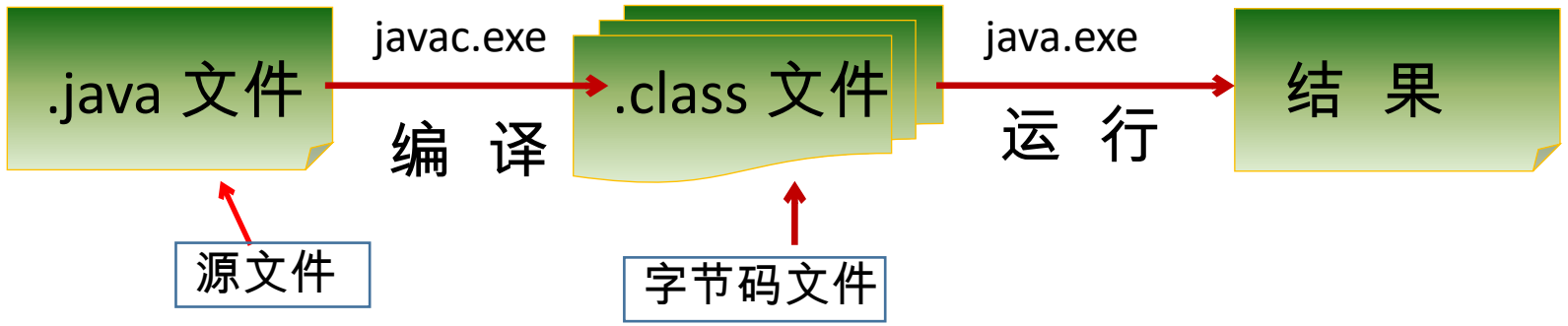
执行命令解析:

以上我们使用了两个命令 **javac** 和 **java**。

第一个 Java 程序 — HelloWorld

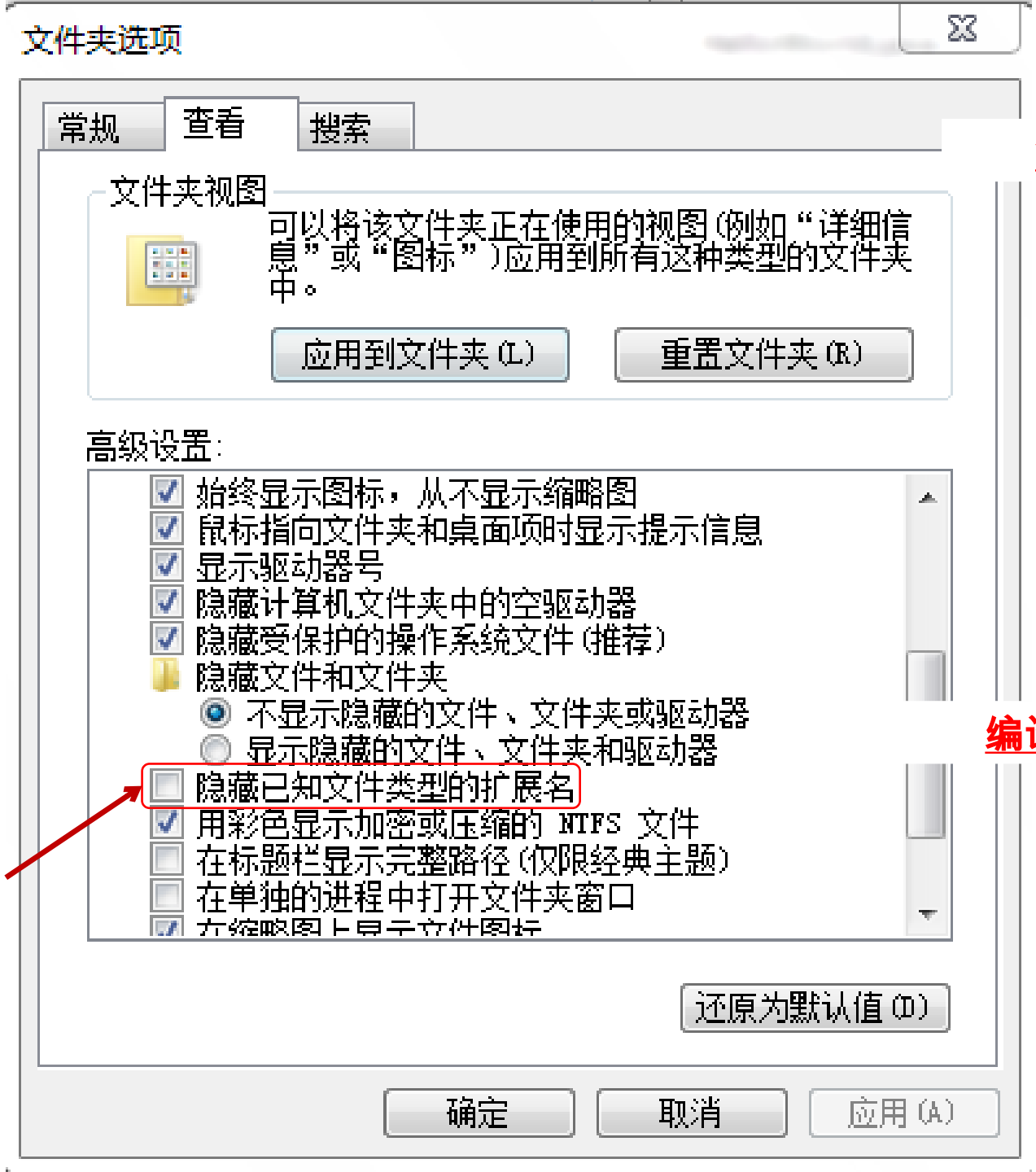
- 步骤：
  1. 将 Java 代码**编写**到扩展名为 .java 的文件中。
  2. 通过 javac 命令对该 java 文件进行**编译**。
  3. 通过 java 命令对生成的 class 文件进行**运行**。

编译和执行过程



- **步骤一：编写**
  - 选择最简单的编辑器：记事本。
  - 敲入代码
    class Test{ }

将文件保存成Test.java ，这个文件是存放 java 代码的文件，称为源文件。



取消勾选

命令行方式

编译和执行过程

● 步骤二：编译

- 有了 java 源文件，通过编译器将其编译成 JVM 可以识别的字节码文件 *.class 。
- 在该源文件目录下，通过 javac 编译工具对 Test.java 文件进行编译。

```
D:\>javac Test.java
```

- 如果程序没有错误，没有任何提示，但在当前目录下会出现一个 Test.class 文件，该文件称为字节码文件，也是可以执行的 java 的程序。

● 步骤三：运行

命令行方式

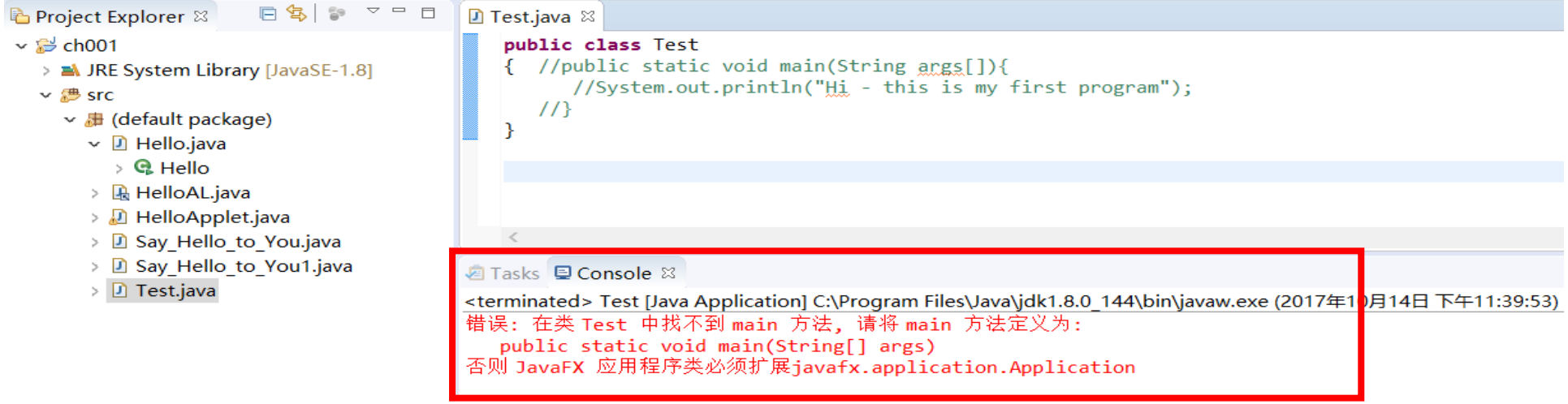
- 有了可执行的 java 程序 (Test.class 字节码文件)
- 通过运行工具 java.exe 对字节码文件进行执行。
- 出现提示：缺少一个名称为 main 的方法。

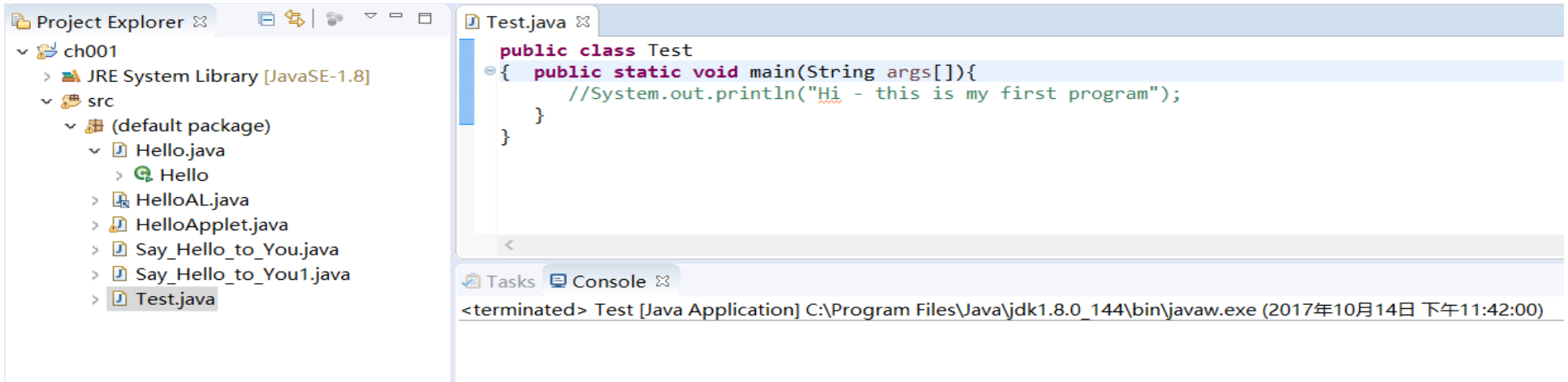
```
D:\>java Test
错误: 在类 Test 中找不到主方法, 请将主方法定义为:
   public static void main(String[] args)
```

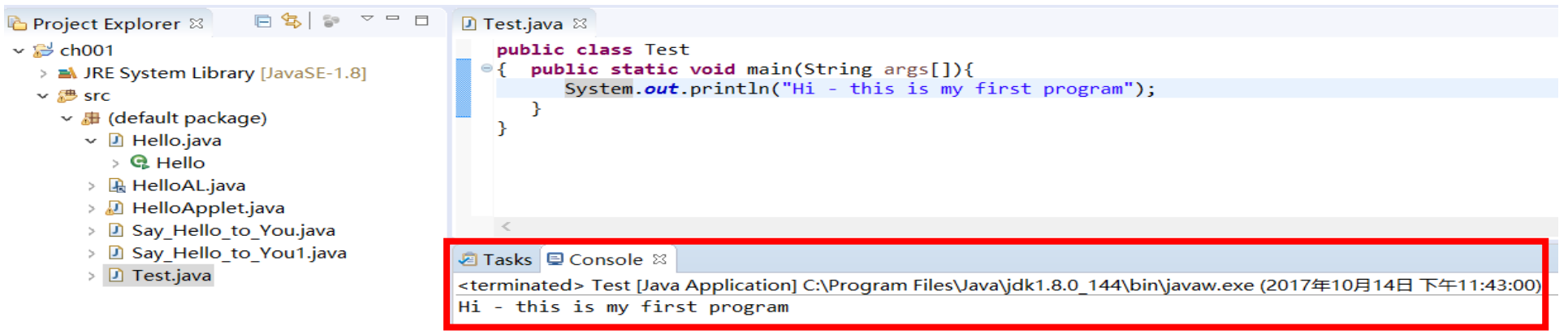
- 因为一个程序的执行需要一个起始点或者入口，所以在 Test 类中的加入 public static void main(String[] args){ }
- 对修改后的 Test.java 源文件需要重新编译，生成新的 class 文件后，再进行执行。

```
D:\>javac Test.java
```

- 发现没有编译失败，但也没有任何效果，因为并没有告诉 JVM 要帮我们做什么事情，也就是没有可以具体执行的语句。
- 想要和 JVM 来个互动，只要在 main 方法中加入一句
- System.out.println(“ Hello World"); 因为程序进行改动，所以再重新编译，运行即可。


Project Explorer
ch001
JRE System Library [JavaSE-1.8]
src
(default package)
Hello.java
Hello
HelloAL.java
HelloApplet.java
Say_Hello_to_You.java
Say_Hello_to_You1.java
Test.java
Test.java
public class Test
{ //public static void main(String args[]){
//System.out.println("Hi - this is my first program");
//}
}
Tasks
Console
<terminated> Test [Java Application] C:\Program Files\Java\jdk1.8.0_144\bin\javaw.exe (2017年10月14日 下午11:39:53)
错误：在类 Test 中找不到 main 方法，请将 main 方法定义为：
public static void main(String[] args)
否则 JavaFX 应用程序类必须扩展javafx.application.Application


Project Explorer
ch001
JRE System Library [JavaSE-1.8]
src
(default package)
Hello.java
Hello
HelloAL.java
HelloApplet.java
Say_Hello_to_You.java
Say_Hello_to_You1.java
Test.java
Test.java
public class Test
{ public static void main(String args[]){
//System.out.println("Hi - this is my first program");
}
}
Tasks
Console
<terminated> Test [Java Application] C:\Program Files\Java\jdk1.8.0_144\bin\javaw.exe (2017年10月14日 下午11:42:00)

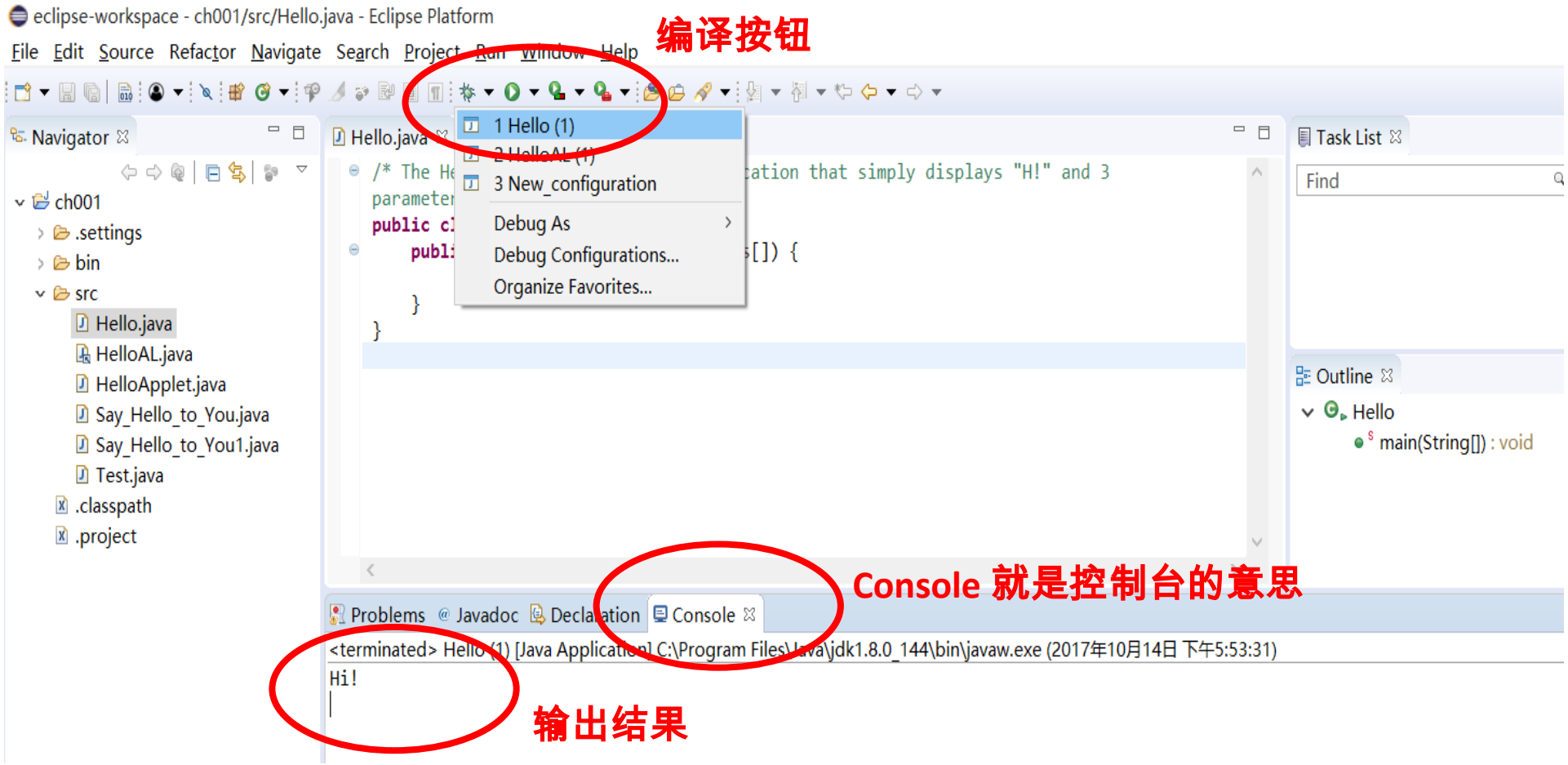

Project Explorer
ch001
JRE System Library [JavaSE-1.8]
src
(default package)
Hello.java
Hello
HelloAL.java
HelloApplet.java
Say_Hello_to_You.java
Say_Hello_to_You1.java
Test.java
Test.java
public class Test
{ public static void main(String args[]){
System.out.println("Hi - this is my first program");
}
}
Tasks
Console
<terminated> Test [Java Application] C:\Program Files\Java\jdk1.8.0_144\bin\javaw.exe (2017年10月14日 下午11:43:00)
Hi - this is my first program

关于应用程序的总结

- Java 源文件以“ java ”为扩展名。源文件的基本组成部分是类（ class ），如下面的 Hello 类。
- 一个源文件中最多只能有一个 public 类。其它类的个数不限，如果**源文件包含一个 public 类，则文件名必须按该类名命名**。
- **Java 应用程序的执行入口是 main() 方法。它有固定的书写格式： public static void main(String[] args) {...}**
- Java 语言严格区分大小写。
- Java 方法由一条条语句构成，每个语句以“ ; ”结束。
- 大括号都是成对出现的，缺一不可。

```
/* The Hello class implements an application that simply displays "H!" and 3
 parameters to the standard output */

public class Hello {
        public static void main(String args[]) {
          System.out.println("Hi!");
        }
}
```


eclipse-workspace - ch001/src/Hello.java - Eclipse Platform
File Edit Source Refactor Navigate Search Project Run Window Help
编译按钮
Navigator
ch001
.settings
bin
src
Hello.java
HelloAL.java
HelloApplet.java
Say_Hello_to_You.java
Say_Hello_to_You1.java
Test.java
.classpath
.project
Hello.java
1 Hello (1)
3 New_configuration
Debug As
Debug Configurations...
Organize Favorites...
Task List
Find
Outline
Hello
main(String[]) : void
Problems
Javadoc
Declaration
Console
<terminated> Hello (1) [Java Application] C:\Program Files\Java\jdk1.8.0_144\bin\javaw.exe (2017年10月14日 下午5:53:31)
Hi!
Console 就是控制台的意思
输出结果

常见问题

```
D:\>javac Test1.java
javac: 找不到文件: Test1.java
用法: javac <options> <source files>
-help 用于列出可能的选项
```

- 源文件名不存在或者写错，或者当前路径错误。

常见问题

```
D:\>java Test1
错误: 找不到或无法加载主类 Test1
```

- 类文件名写错，或者类文件不在当前路径下，或者**不在classpath 指定路径下。**

常见问题

常见问题

```
D:\>javac Test.java
Test.java:1: 错误: 类Test1是公共的，应在名为 Test1.java 的文件中声明
public class Test1{
       ^
1 个错误
```

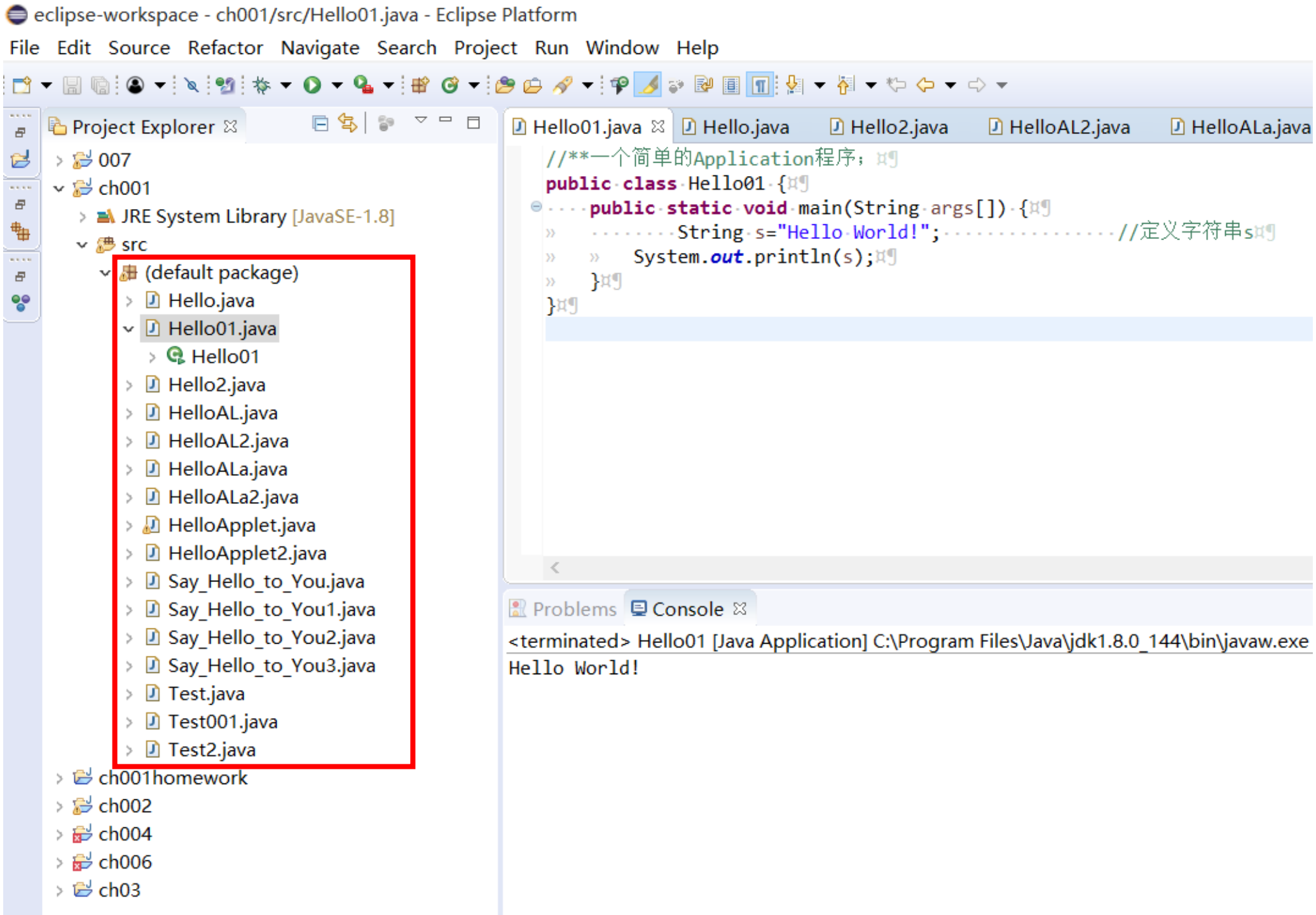
- 声明为 public 的主类应与文件名一致，否则编译失败

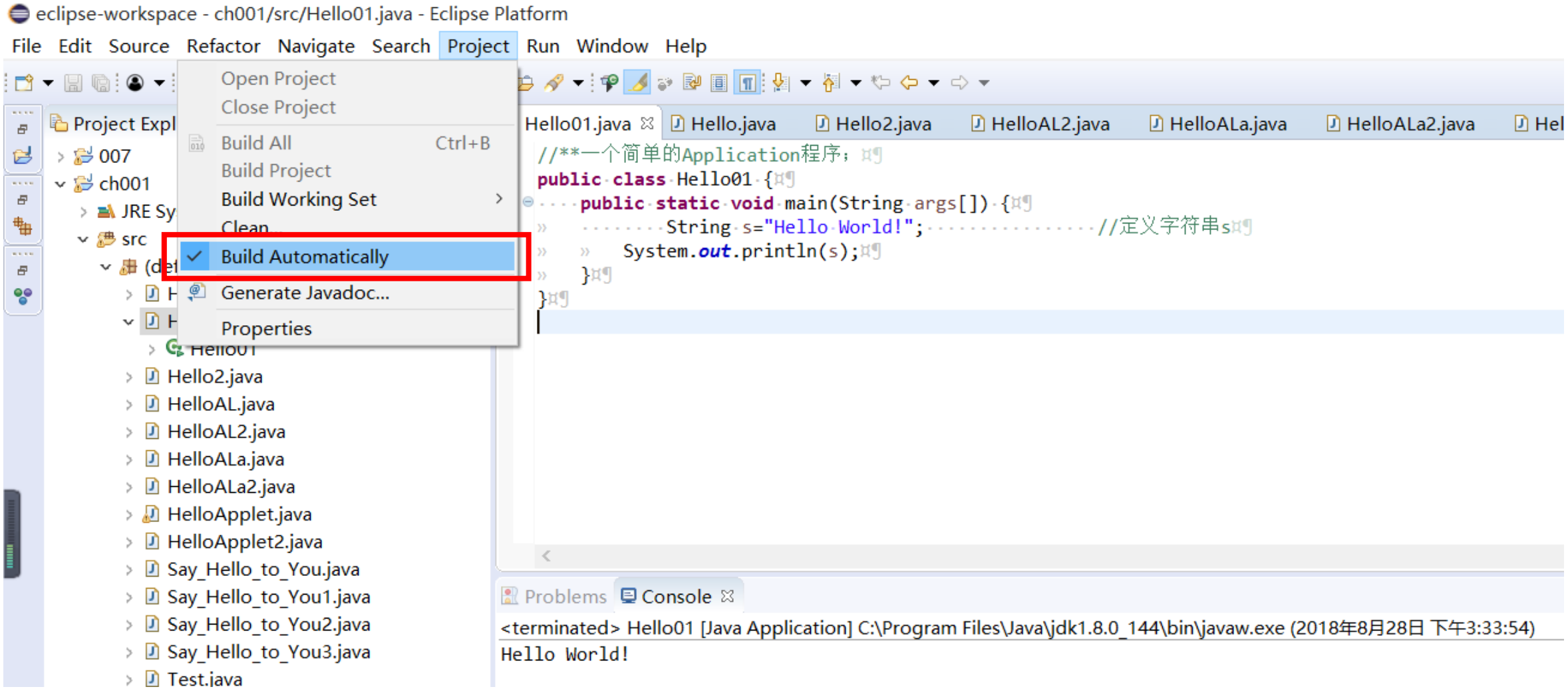
常见问题

```
D:\>javac Test.java
Test.java:3: 错误: 需要';'
                System.out.println("hello")
                                          ^
1 个错误
```

- 编译失败，注意错误出现的行数，再到源代码中指定位置改错

常见问题


eclipse-workspace - ch001/src/Hello01.java - Eclipse Platform
File Edit Source Refactor Navigate Search Project Run Window Help
Project Explorer
007
ch001
JRE System Library [JavaSE-1.8]
src
(default package)
Hello.java
Hello01.java
Hello01
Hello2.java
HelloAL.java
HelloAL2.java
HelloALa.java
HelloALa2.java
HelloApplet.java
HelloApplet2.java
Say_Hello_to_You.java
Say_Hello_to_You1.java
Say_Hello_to_You2.java
Say_Hello_to_You3.java
Test.java
Test001.java
Test2.java
ch001homework
ch002
ch004
ch006
ch03
Hello01.java
Hello.java
Hello2.java
HelloAL2.java
HelloALa.java
//**一个简单的Application程序；
public class Hello01 {
public static void main(String args[]) {
String s="Hello World!";//定义字符串s
System.out.println(s);
}
}
Problems
Console
<terminated> Hello01 [Java Application] C:\Program Files\Java\jdk1.8.0_144\bin\javaw.exe
Hello World!


eclipse-workspace - ch001/src/Hello01.java - Eclipse Platform
File Edit Source Refactor Navigate Search Project Run Window Help
Open Project
Close Project
Build All Ctrl+B
Build Project
Build Working Set
Clean...
Build Automatically
Generate Javadoc...
Properties
Project Expl
007
ch001
JRE Sy
src
Hello01
Hello2.java
HelloAL.java
HelloAL2.java
HelloALa.java
HelloALa2.java
HelloApplet.java
HelloApplet2.java
Say_Hello_to_You.java
Say_Hello_to_You1.java
Say_Hello_to_You2.java
Say_Hello_to_You3.java
Test.java
Hello01.java
Hello.java
Hello2.java
HelloAL2.java
HelloALa.java
HelloALa2.java
//**一个简单的Application程序；
public class Hello01 {
public static void main(String args[]) {
String s="Hello World!"; //定义字符串s
System.out.println(s);
}
}
Problems
Console
<terminated> Hello01 [Java Application] C:\Program Files\Java\jdk1.8.0_144\bin\javaw.exe (2018年8月28日 下午3:33:54)
Hello World!

注 释

- 用于注解说明解释程序的文字就是注释。
- 提高了代码的阅读性；调试程序的重要方法。
- Java 中的注释类型：
  - 单行注释　// --　<!-- -->
  - 多行注释
  - 文档注释（ java 特有 ）
- 注释是一个程序员必须要具有的良好编程习惯。
- 将自己的思想（**设计**）通过注释先整理出来，再用代码去体现

注释

注 释

- **单行注释**
  - 格式： // 注释文字
- **多行注释**
  - 格式： /* 注释文字 */

注释

- 注：
  - 对于单行和多行注释，被注释的文字，不会被 JVM（ java 虚拟机）解释执行。
  - 多行注释里面不允许有多行注释嵌套。

注 释

- **文档注释（ java 特有）**
  - **格式： /****

    *** @author 指定 java 程序的作者**
    ***@version 指定源文件的版本**
    ***@param 方法的参数说明信息**
    ***/**

- 注释内容可以被 JDK 提供的工具 javadoc 所解析，生成一套以网页文件形式体现的该程序的说明文档。

- 操作方式

```
D:\javase\code\unit1>javadoc -d mydoc -author -version HelloWorld.java
```

Java API 文档

API 文档

- API （ Application Programming Interface, 应用程序编程接口 ）是 Java 提供的基本编程接口。
- Java 语言提供了大量的基础类，因此 Oracle 也为这些基础类提供了相应的 API 文档，用于告诉开发者如何使用这些类，以及这些类里包含的方法。
- 下载 API ： http://www.oracle.com/technetwork/java/javase/downloads/index.html
  - Additional Resources-Java SE 7 Documentation 下载。

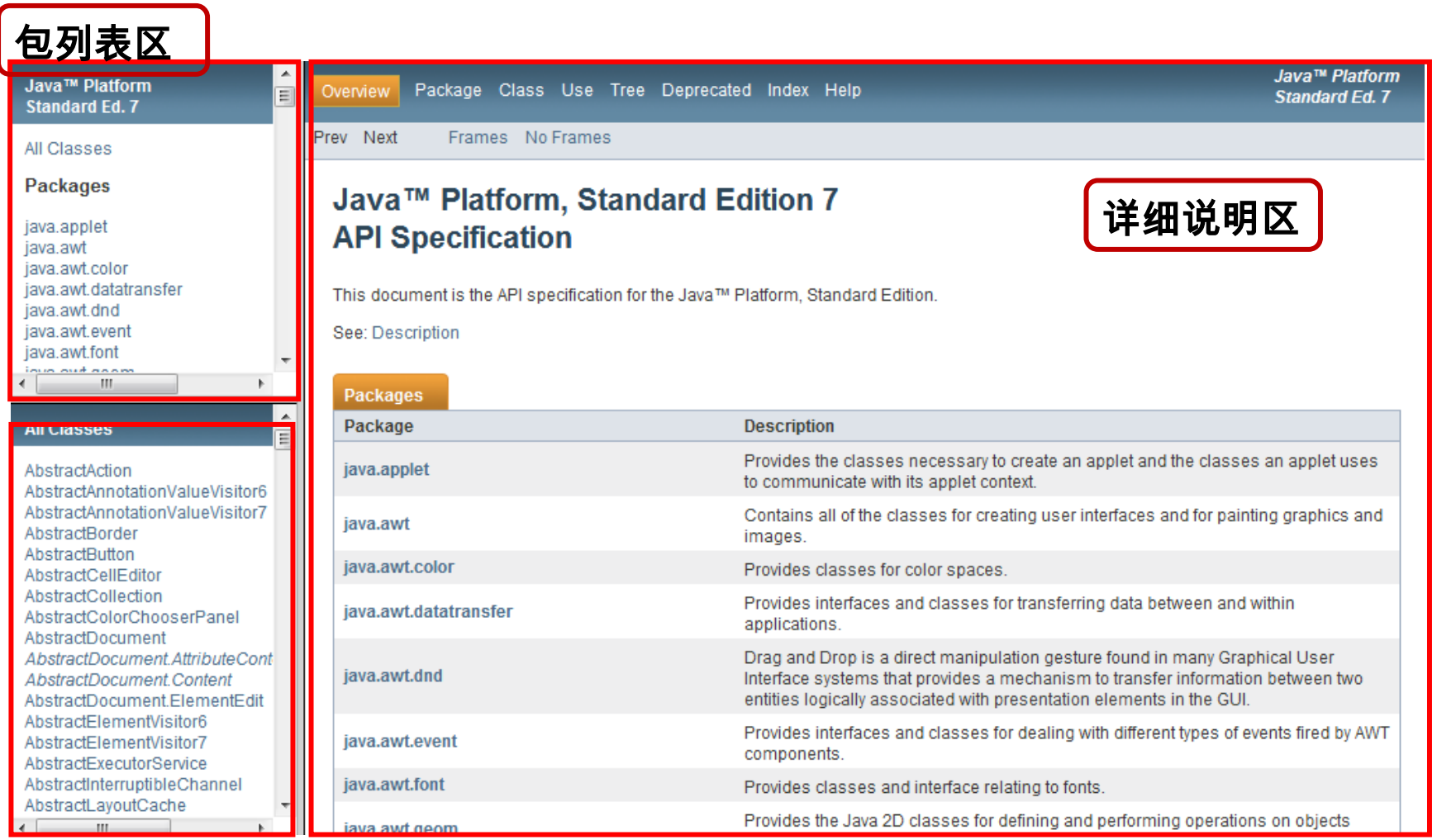
Java API 文档

API 文档

包列表区

类列表区

详细说明区



Java 程序的两种常见类型

JAVA 程序类型

- **Java Application（ Java 应用程序）**：是一个完整的应用程序，可以独立地运行
  - 运行在 Java 虚拟机 (JVM) 上
  - 中间代码
  - 必须有 main 函数
- **Java Applet（小应用程序）**：不是一个完整的应用程序，而是框架程序中的一个模块，所以只能在 WWW 浏览器环境下运行
  - 可以没有 main 函数
  - 必须由某个支持 java 的浏览器来运行

Java Application 程序的例子

Application

```
/** 一个简单的 Application 程序 HelloWorld.java ;
*/
public class HelloWorld {
    public static void main(String args[]) {
            String s="HelloWorld!";              // 定义字符串 s
                System.out.println(s);
     }
}
```

(1)Java 语言标识符的字母区分大小写；

(2) 一个程序可以由一个或多个类组成，其中必须有也只能有一个主类。

(3) 源文件名必须与程序的主类名一致，并且以 .java 为其后缀。

Application



程序说明：

Applicatio
n

- 用保留字 class 来声明一个新的类，其类名为 HelloWorld ，它是一个公共类 (public) 。 一个 Java 程序中可以定义多个类，**但只有一个类可以定义为 public 类**；
- 在该类中定义了一个 **main( ) 方法**，其修饰符有：

public 表示访问权限，指明所有的类都可以使用这一方法；

static 指明该方法是一个类方法，它可以通过类名直接调用；

void 则指明 main( ) 方法不返回任何值。

- main( ) 是执行程序的入口；对于一个可执行的应用程序来说，main( ) 方法是必需的，而且必须按照如上的格式来定义。
- main( ) 方法括号 ( ) 中的 String args [ ] 是传递给 main( ) 方法的参数，参数可以没有或多个。

应用程序的编译执行过程

Application

- Java 引入编译的主要原因：

(1) 保证程序的语法正确性；

(2) 通过编译引入 Java 类库中已编写好的模块；

(3) 编译成功的结果将生成一个或多个字节码文件，每个字节码文件对应源程序中定义的一个类，该文件的名字是它所对应的类的名字，并以 .class 为统一的后缀名。

编译一个源程序的命令为：

C:\>javac HelloWorld.java

- Java 的源程序经过编译后，每个类都生成一个类文件

应用程序的编译执行过程

Application

执行一个 Java 程序的命令格式为：

C:\>java HelloWorld

- 该命令调用 Java 的解释器软件 java.exe 运行编译好的 Java 字节码程序。程序执行结果显示： HelloWorld!

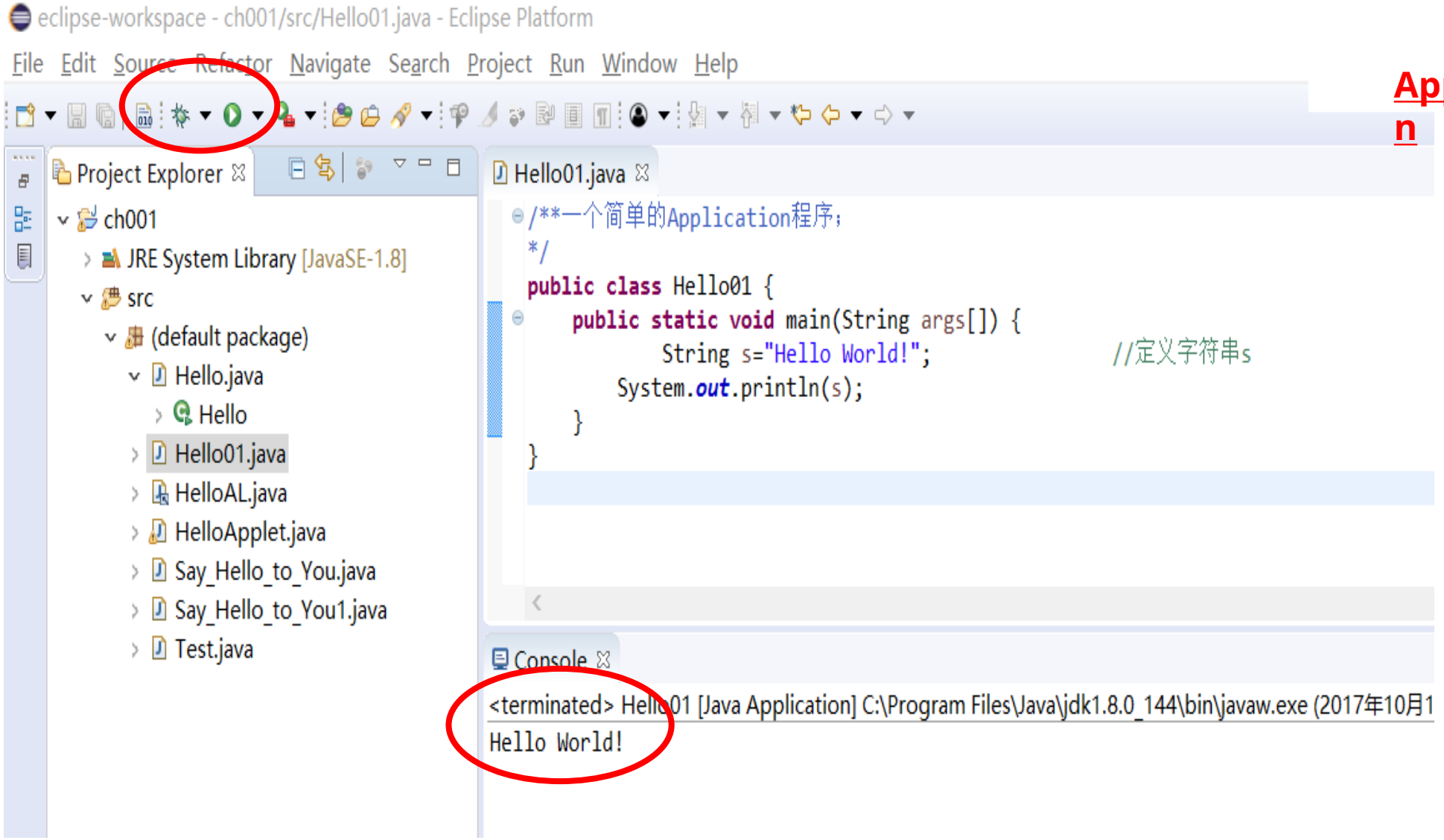
•解释运行的三个阶段：

(1) 载入： Java 解释器中的类载入器将字节码文件加载到内存 (网上运行程序则通过网络下载到本地内存) ；

(2) 代码校验： Java 解释器中的代码检验器检查这些字节码的合法性；

(3) 解释执行：合法的字节码程序由 **Java 解释器逐句地解释运行**。

Application



一个简单的 Applet 程序

Applet

- Applet 程序是一个嵌入在浏览器中执行的 Java 程序，它必须扩展 Applet 类。扩展 Applet 类的程序没有构造方法，也不需要 main() 方法。

- 一个 Applet 程序一般包含 4 个重要方法： init() 、 start() 、 stop() 和 destroy() ，它的执行由 init() 方法控制。 Applet 程序一旦装载后，init() 方法只执行一次。每次重新访问 Applet 时，则要调 start() 方法。每次 Applet 所在的 Web 页面被其他页面所取代时，则要调 stop() 方法。

一个简单的 Applet 程序

Applet

```
/* 这是一个 Applet 程序；
*/
import java.awt.*;
import java.applet.*;
public class HelloApp extends Applet {
    public void paint( Graphics g ) {
            g.drawString("HelloWorld!",20 ,
    20);
       }
}
```

程序中，首先用 import 语句输入 java.awt 和 java.applet 包中所有的类，使得该程序可能使用这些包中所定义的类，它类似于 C 中的 #include 语句。

程序说明：

```
import java.awt.*;
import java.applet.*;
public class HelloApp extends Applet
  {
  public void paint( Graphics g ) {
            g.drawString("HelloWorld!",
                          20 , 20);
     }
}
```

Applet

接着声明一个公共类 HelloApp ，用 extends 指明它是 Applet 的子类。在类中，我们**重写父类 Applet 的 paint( ) 方法，其中参数 g 为 Graphics 类，它表明当前画图的上下文**。

在 paint() 方法中，调用图形对象 g 的方法 drawString() ，在坐标 (20,20) 处输出字符串“ Hello World! ”。

JavaApplet 基本结构

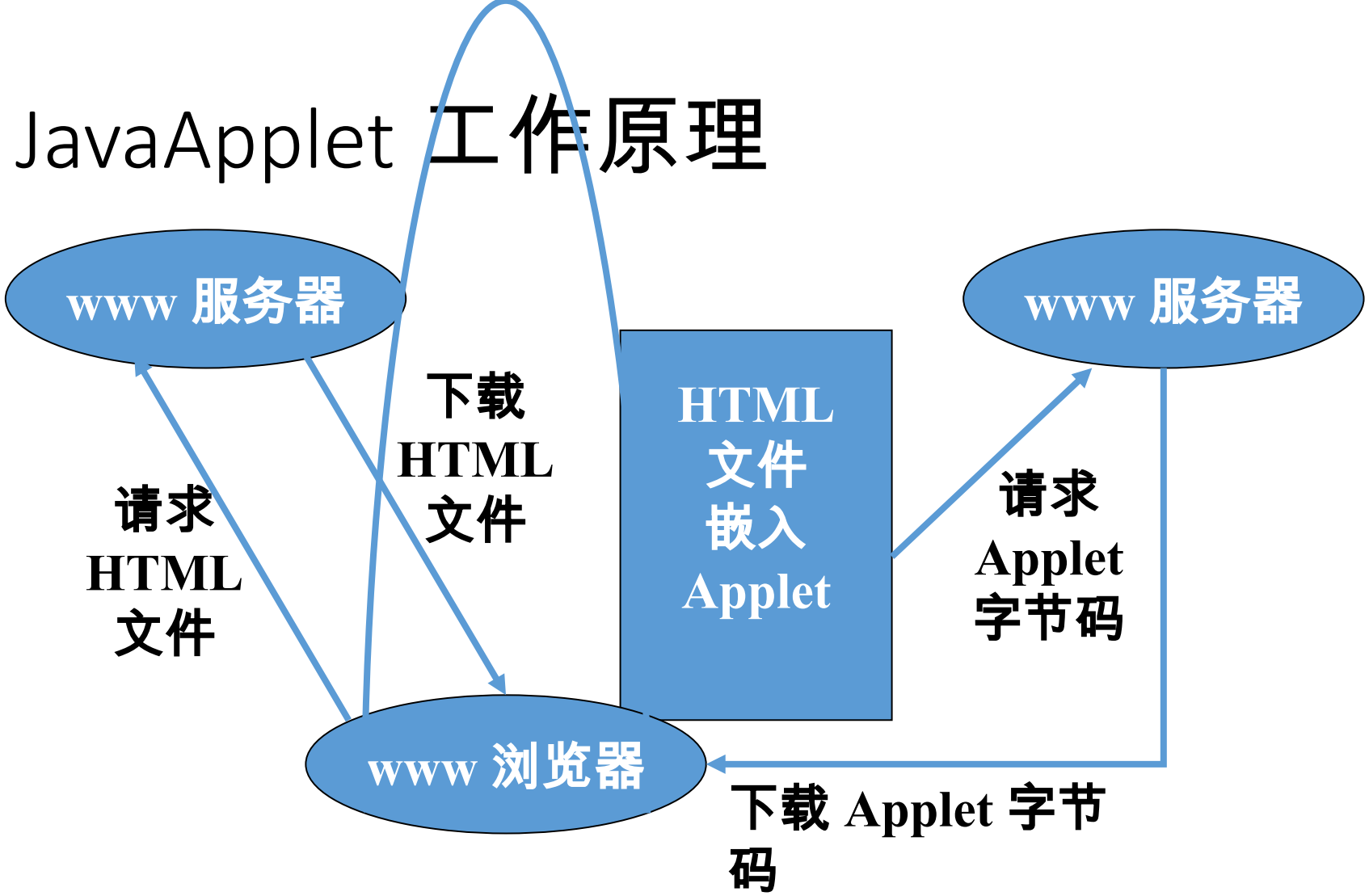
Apple
t

```
import java.applet.*;
import java.awt.*;

class  类名称  extends Applet
{
        类成员声明
}
```

JavaApplet 工作原理

Apple
t



解释执行 Applet 字节码

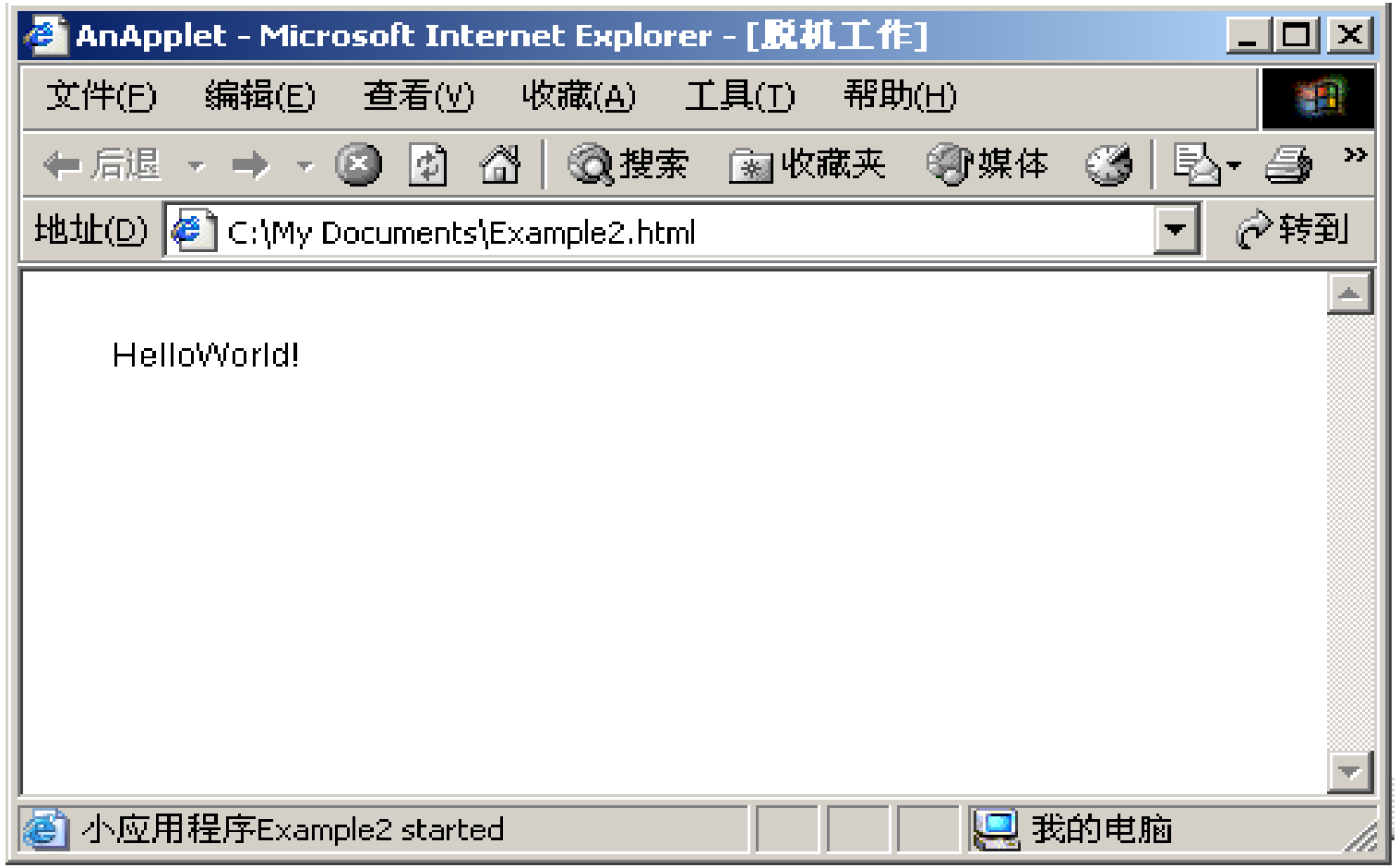
嵌入到 HTML 文件中

Applet

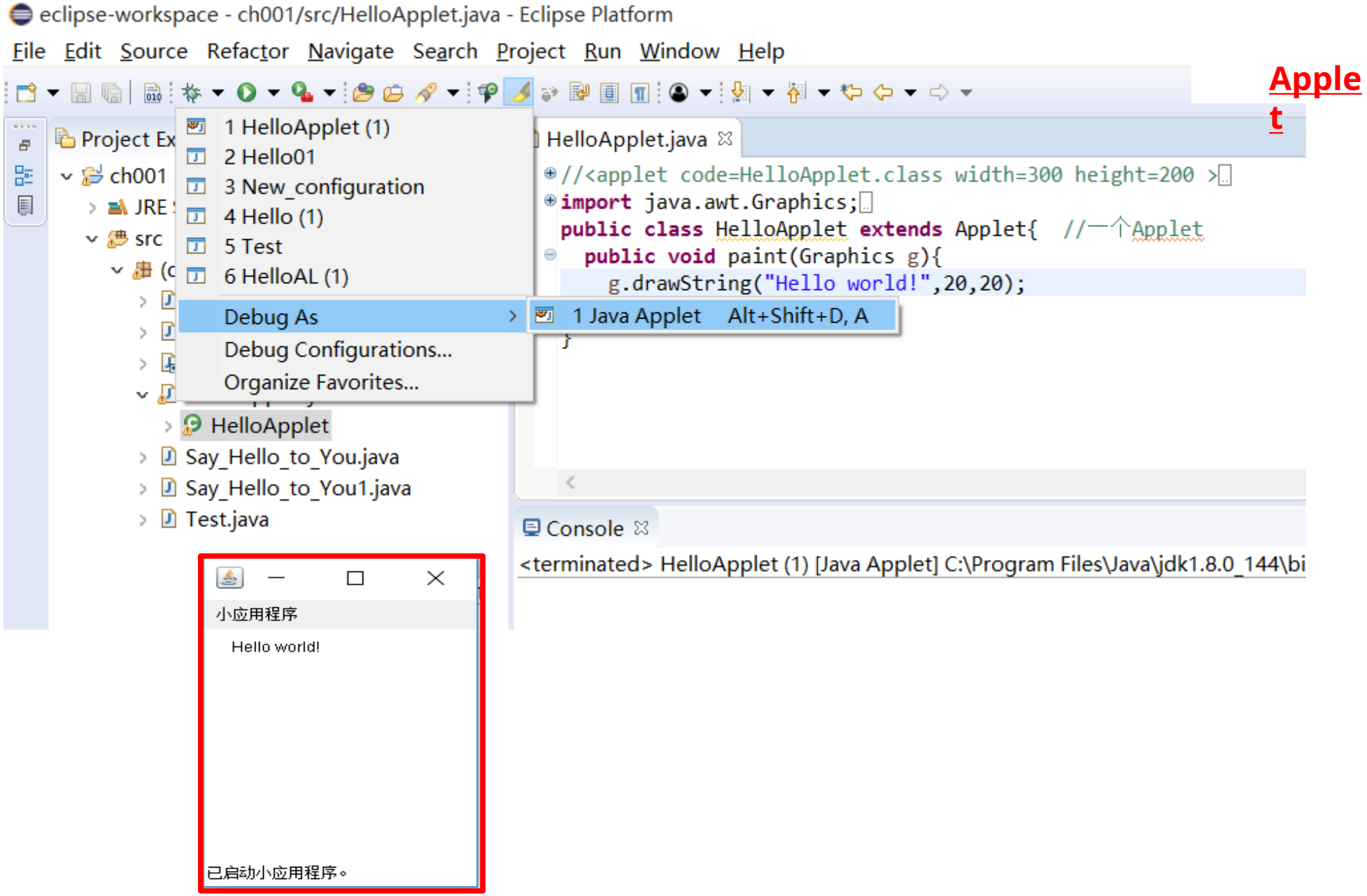
```
<HTML>
<HEAD>
<TITLE>AnApplet</TITLE>
</HEAD>
<BODY>
  <APPLET    Code="HelloApp.class"
                Width=200    Height=40>
  </APPLET>
</BODY>
</HTML>
```

浏览效果

Apple
t


AnApplet - Microsoft Internet Explorer - [脱机工作]
文件(F) 编辑(E) 查看(V) 收藏(A) 工具(T) 帮助(H)
后退 搜索 收藏夹 媒体
地址(D) C:\My Documents\Example2.html
转到
HelloWorld!
小应用程序Example2 started
我的电脑

Applet


eclipse-workspace - ch001/src/HelloApplet.java - Eclipse Platform
File Edit Source Refactor Navigate Search Project Run Window Help
1 HelloApplet (1)
2 Hello01
3 New_configuration
4 Hello (1)
5 Test
6 HelloAL (1)
Debug As
Debug Configurations...
Organize Favorites...
1 Java Applet Alt+Shift+D, A
Project Ex
ch001
JRE
src
HelloApplet
Say_Hello_to_You.java
Say_Hello_to_You1.java
Test.java
HelloApplet.java
//<applet code=HelloApplet.class width=300 height=200 >
import java.awt.Graphics;
public class HelloApplet extends Applet{ //一个Applet
public void paint(Graphics g){
g.drawString("Hello world!",20,20);
}
Console
<terminated> HelloApplet (1) [Java Applet] C:\Program Files\Java\jdk1.8.0_144\bi
小应用程序
Hello world!
已启动小应用程序。

具有输入功能的程序

- 前面介绍的程序都只具有输出功能，如果想对 Java 整体有一个大概的了解，可能很想看看具有输入功能的程序。这里也顺便提到 Java 的异常处理机制，这也是 Java 的特色之一。
- **输入比输出更容易出错。一个错误的输入将会导致程序的崩溃（程序中断或退出），运行时发生的错误称为异常**，Java 提供了特殊的机制来处理异常。**一个最简单的处理输入 / 输出异常的方法是在 main() 方法后面加上“ throws IOException ”**。

通过键盘读入信息的程序

具有输入功能的程序

交互式输入输出。这个程序首先提示用户输入姓名，然后等待用户输入。当用户输入姓名，并按 Enter 键后，系统很快给出一个问候信息。

```
import  java.io.*;  // 调用 IO 类库
  public  class  Say_Hello_to_You{
     public  static  void  main (String args[]) throws IOException{
//reader 定义为 InputStreamReader 类的一个实例，并把系统的输入流对象
//System.in 与作为它的输入流，把数据从键盘传递到程序当中
        InputStreamReader  reader=new  InputStreamReader(System.in);
//将 input 对象定义为 BufferedReader 类的实例，并把 reader 对象作为它的输入流
        BufferedReader  input=new  BufferedReader(reader);
        System.out.print("Enter  your  name:");
//input 对象的 readLine() 方法从键盘读取一整行的字符，把它们当做 String 对象来对待
        String  name =input.readLine();
// 打印  name 中的字符串
        System.out.println("Hello,"+ name + "!");
     }
  }
```

通过键盘读入信息的程序

- 最后执行该程序，其交互方式如图所示。



- 程序简单分析：
- 源代码文件中的第 1 行是：

  import java.io.*;

- 它告诉编译器到 java.io 类库中寻找程序中用到的 I/O 类。事实上该程序用到 3 个 I/O 类： IOException 、InputStreamReader 和 BufferedReader 类。

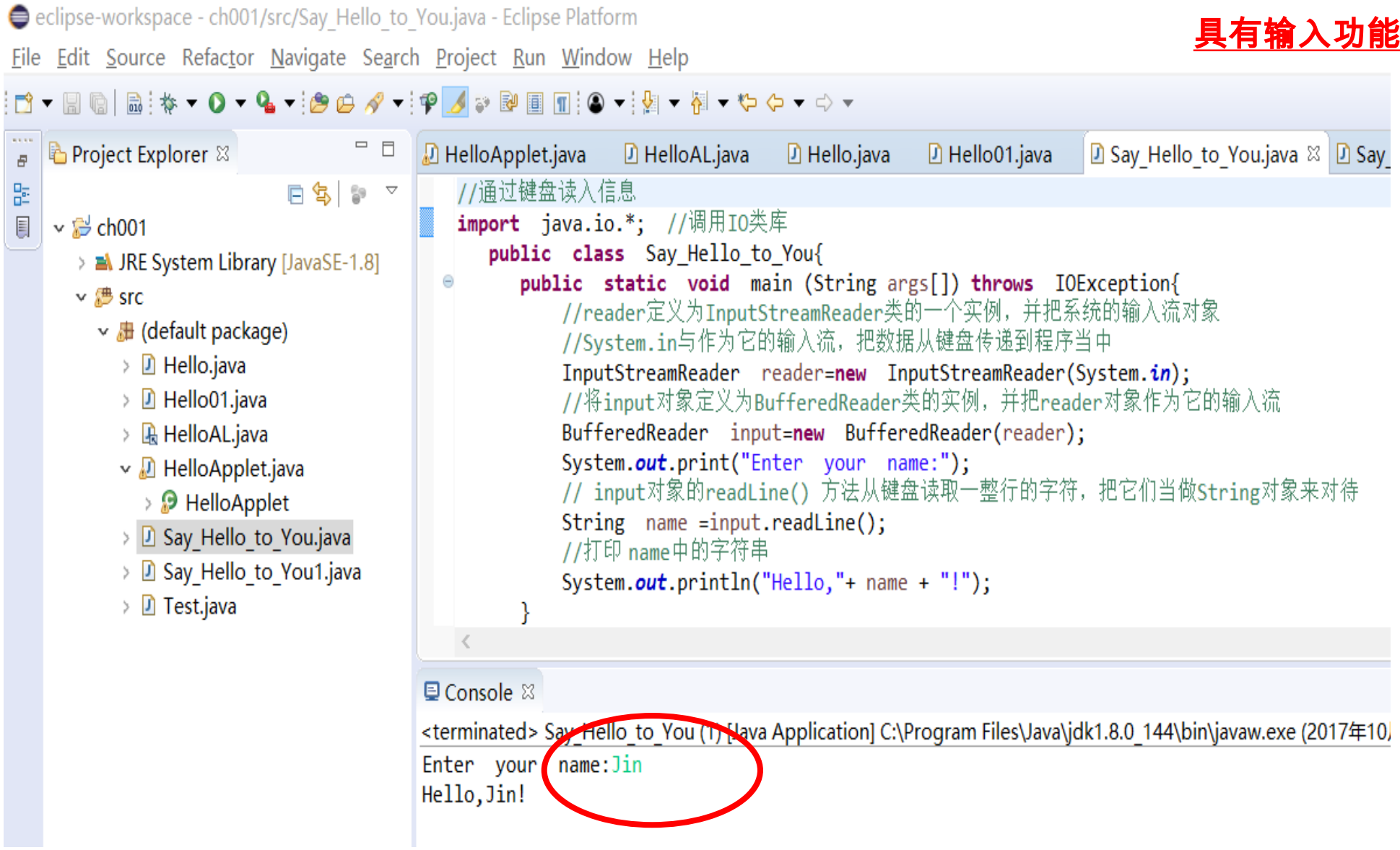
通过键盘读入信息的程序

具有输入功能的程序

- 第 4 行中，把 reader 定义为 InputStreamReader 类的一个实例，并把系统的输入流对象 System.in 与作为它的输入流，把数据从键盘传递到程序当中。
- 第 5 行中，把 input 对象定义为 BufferedReader 类的一个实例，并把 reader 对象作为它的输入流。
- 第 7 行 , input 对象的 readLine() 方法从键盘读取一整行的字符，并把它们当做 String 对象来对待：

  String name=input.readline();
- 该语句声明了一个 String 对象 name ，它被初始化为 input.readLine() 方法的返回值。无论用户从键盘输入了什么字符，它都被保存在 name 对象中。
- name 中的字符串被接下来的语句打印出来。

具有输入功能的程序

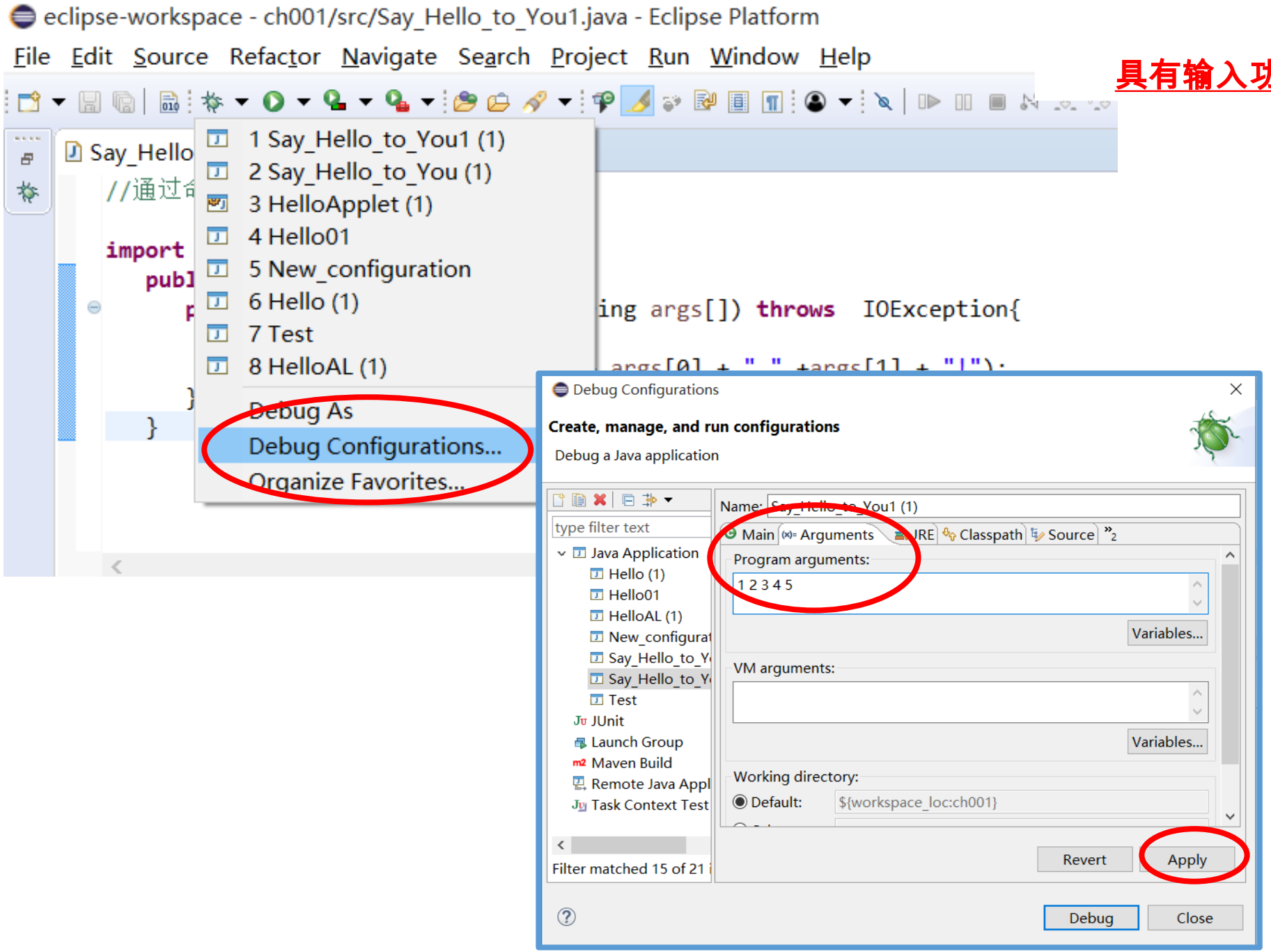

eclipse-workspace - ch001/src/Say_Hello_to_You.java - Eclipse Platform
File Edit Source Refactor Navigate Search Project Run Window Help
Project Explorer
ch001
JRE System Library [JavaSE-1.8]
src
(default package)
Hello.java
Hello01.java
HelloAL.java
HelloApplet.java
HelloApplet
Say_Hello_to_You.java
Say_Hello_to_You1.java
Test.java
HelloApplet.java
HelloAL.java
Hello.java
Hello01.java
Say_Hello_to_You.java
//通过键盘读入信息
import java.io.*; //调用IO类库
public class Say_Hello_to_You{
public static void main (String args[]) throws IOException{
//reader定义为InputStreamReader类的一个实例，并把系统的输入流对象
//System.in与作为它的输入流，把数据从键盘传递到程序当中
InputStreamReader reader=new InputStreamReader(System.in);
//将input对象定义为BufferedReader类的实例，并把reader对象作为它的输入流
BufferedReader input=new BufferedReader(reader);
System.out.print("Enter your name:");
// input对象的readLine() 方法从键盘读取一整行的字符，把它们当做String对象来对待
String name =input.readLine();
//打印 name中的字符串
System.out.println("Hello,"+ name + "!");
}
Console
<terminated> Say_Hello_to_You (1) [Java Application] C:\Program Files\Java\jdk1.8.0_144\bin\javaw.exe (2017年10
Enter your name:Jin
Hello,Jin!

通过命令行参数输入信息

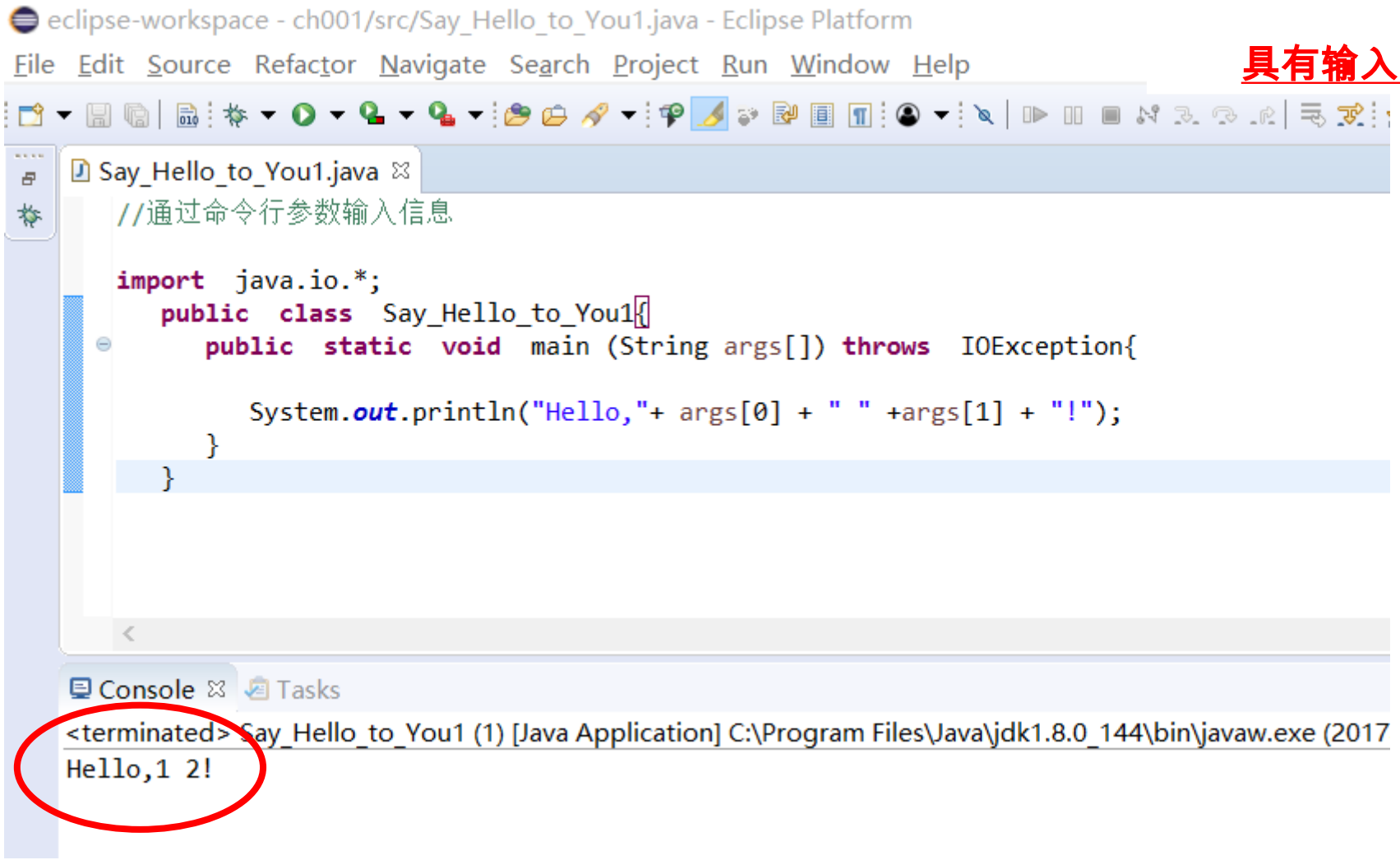
具有输入功能的程序

```
// 通过命令行参数输入信息
import  java.io.*;
   public  class  Say_Hello_to_You1{
      public  static  void  main (String args[]) throws  IOException{
         System.out.println("Hello,"+ args[0] + " " +args[1] + "!");
      }
   }
```

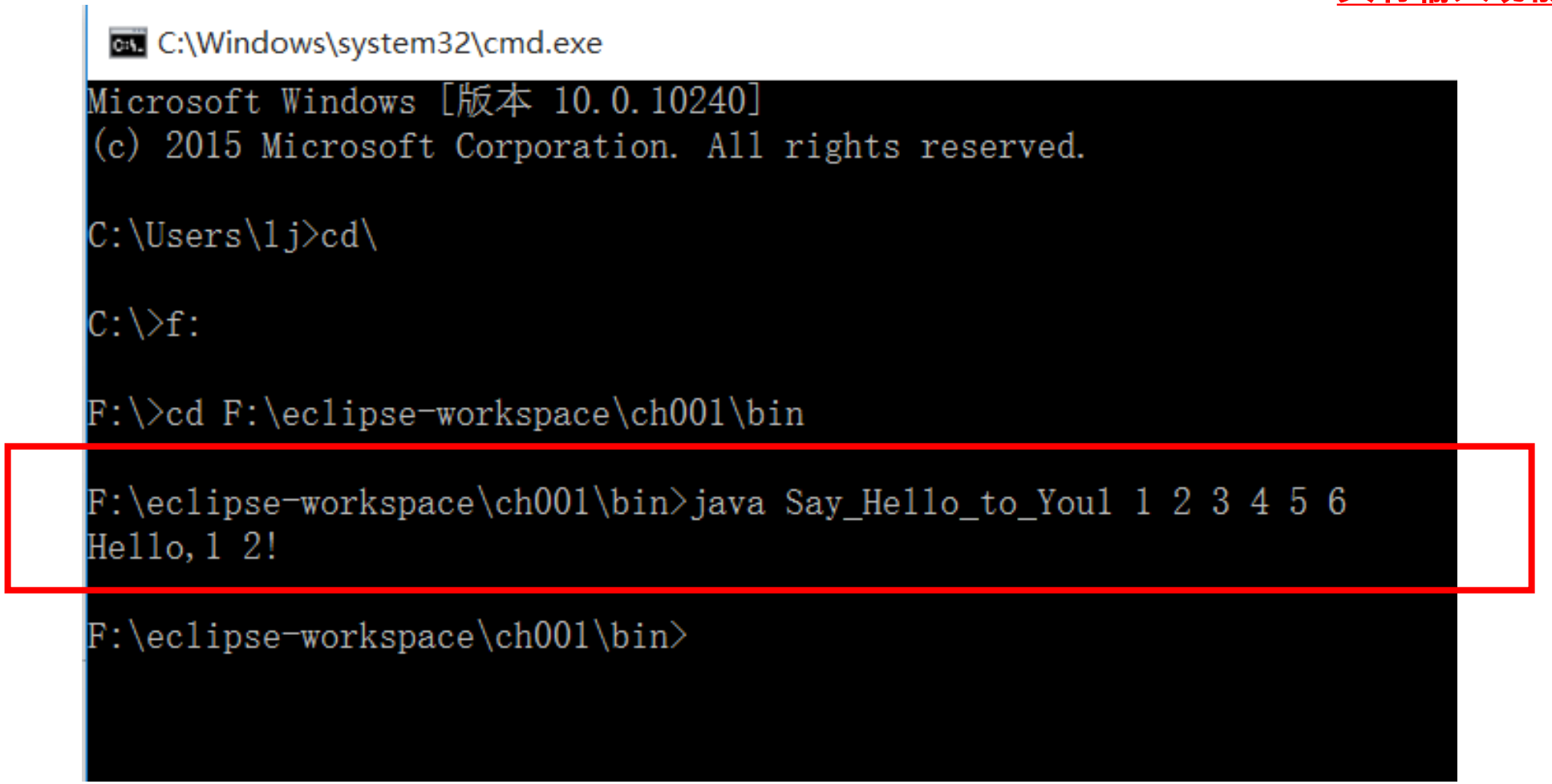
具有输入功能的程序


eclipse-workspace - ch001/src/Say_Hello_to_You1.java - Eclipse Platform
File Edit Source Refactor Navigate Search Project Run Window Help
1 Say_Hello_to_You1 (1)
2 Say_Hello_to_You (1)
3 HelloApplet (1)
4 Hello01
5 New_configuration
6 Hello (1)
7 Test
8 HelloAL (1)
Debug As
Debug Configurations...
Organize Favorites...
Debug Configurations
Create, manage, and run configurations
Debug a Java application
type filter text
Java Application
Hello (1)
Hello01
HelloAL (1)
Test
JUnit
Launch Group
Maven Build
Remote Java Appl
Task Context Test
Filter matched 15 of 21
Name:
Main
Arguments
JRE
Classpath
Source
Program arguments:
1 2 3 4 5
Variables...
VM arguments:
Variables...
Working directory:
Default:
${workspace_loc:ch001}
Revert
Apply
Debug
Close

具有输入功能的程序


eclipse-workspace - ch001/src/Say_Hello_to_You1.java - Eclipse Platform
File Edit Source Refactor Navigate Search Project Run Window Help
Say_Hello_to_You1.java
//通过命令行参数输入信息
import java.io.*;
public class Say_Hello_to_You1{
public static void main (String args[]) throws IOException{
System.out.println("Hello,"+ args[0] + " " +args[1] + "!");
}
}
Console
Tasks
<terminated> Say_Hello_to_You1 (1) [Java Application] C:\Program Files\Java\jdk1.8.0_144\bin\javaw.exe (2017
Hello,1 2!

具有输入功能的程序


C:\Windows\system32\cmd.exe
Microsoft Windows [版本 10.0.10240]
(c) 2015 Microsoft Corporation. All rights reserved.
C:\Users\lj>cd\
C:\>f:
F:\>cd F:\eclipse-workspace\ch001\bin
F:\eclipse-workspace\ch001\bin>java Say_Hello_to_You1 1 2 3 4 5 6
Hello,1 2!
F:\eclipse-workspace\ch001\bin>

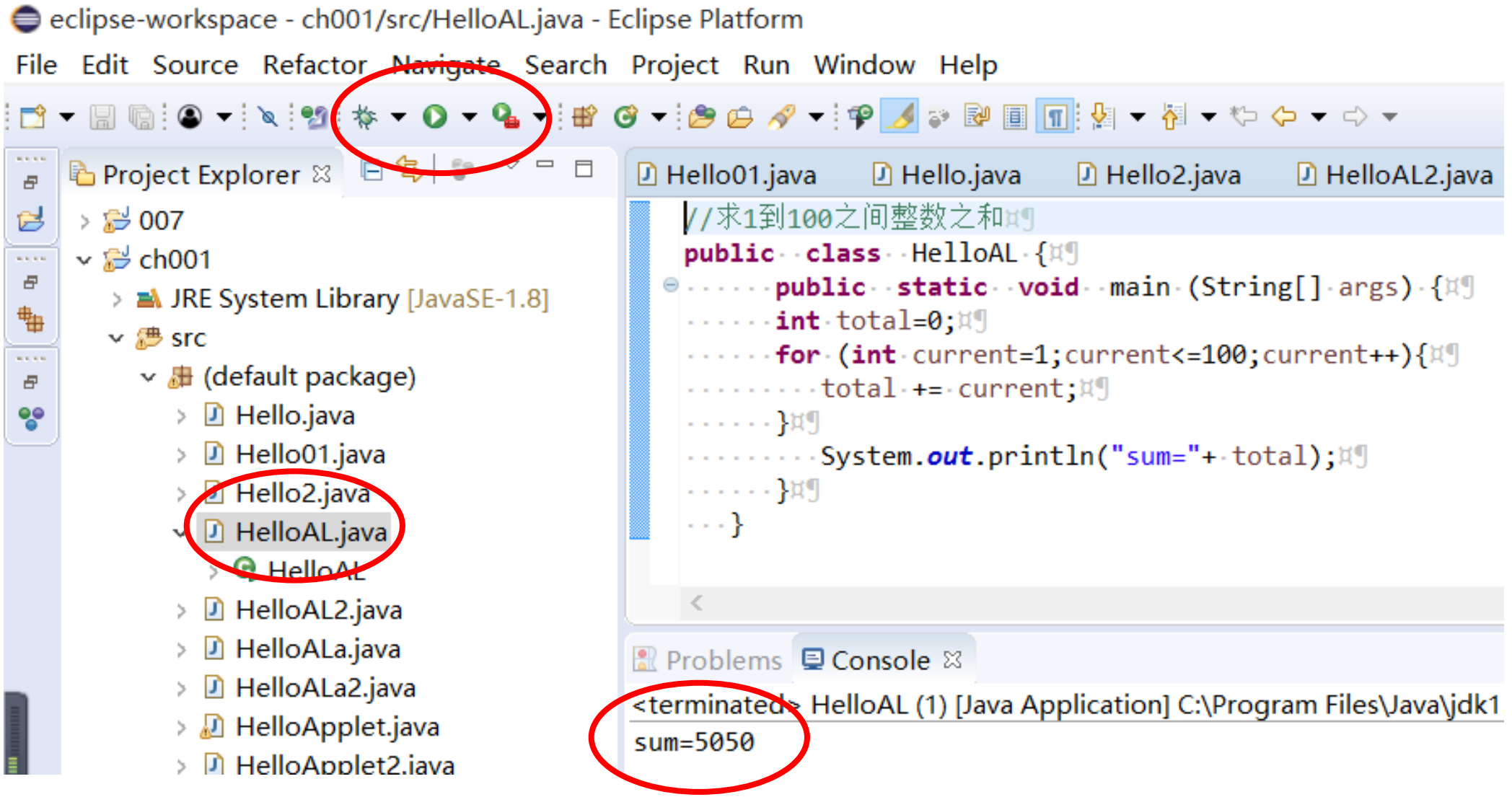
一个求和的程序示例

具有输出功能的程序

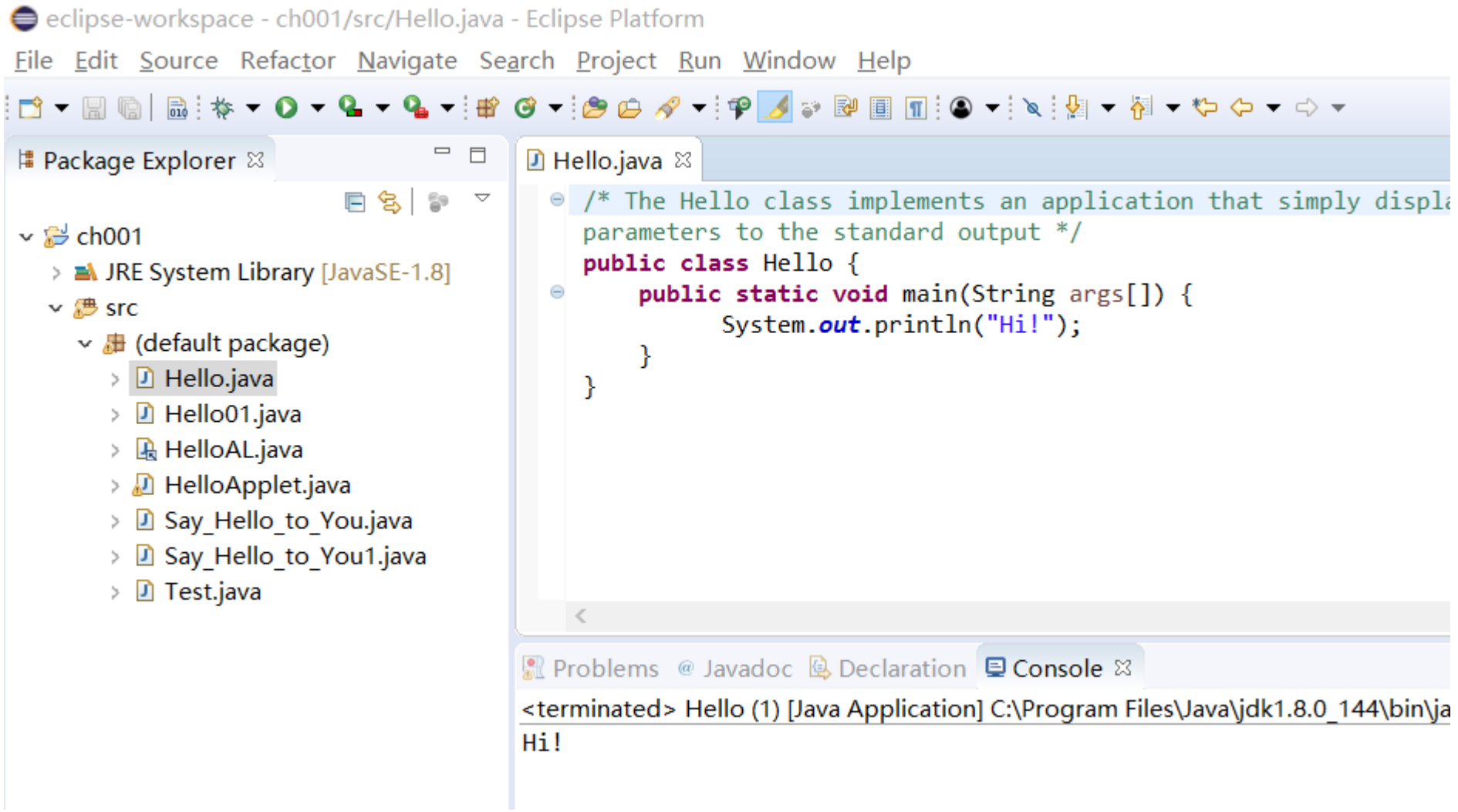
```
// 求 1 到 100 之间整数之和
public  class  HelloAL {
    public  static  void  main (String[] args) {
    int total=0;
    for (int current=1;current<=100;current++){
      total += current;
    }
      System.out.println("sum="+ total);
    }
  }
```

一个求和的程序示例

具有输出功能的程序



具有输出功能的程序


eclipse-workspace - ch001/src/Hello.java - Eclipse Platform
File Edit Source Refactor Navigate Search Project Run Window Help
Package Explorer
ch001
JRE System Library [JavaSE-1.8]
src
(default package)
Hello.java
Hello01.java
HelloAL.java
HelloApplet.java
Say_Hello_to_You.java
Say_Hello_to_You1.java
Test.java
Hello.java
/* The Hello class implements an application that simply displa
parameters to the standard output */
public class Hello {
public static void main(String args[]) {
System.out.println("Hi!");
}
}
Problems Javadoc Declaration Console
<terminated> Hello (1) [Java Application] C:\Program Files\Java\jdk1.8.0_144\bin\ja
Hi!

回顾

知识回顾

- JDK,JRE,JVM 的关系。
- 环境变量 path 配置及其作用。
- Java 程序的编写、编译、运行步骤。

```
E:\_Work\Java\sample>javac HelloWorld.java

E:\_Work\Java\sample>java HelloWorld
Hello World
```

- Java 程序编写的规则。
- 在配置环境、编译、运行各个步骤中常见的错误以及解决方法。

小结

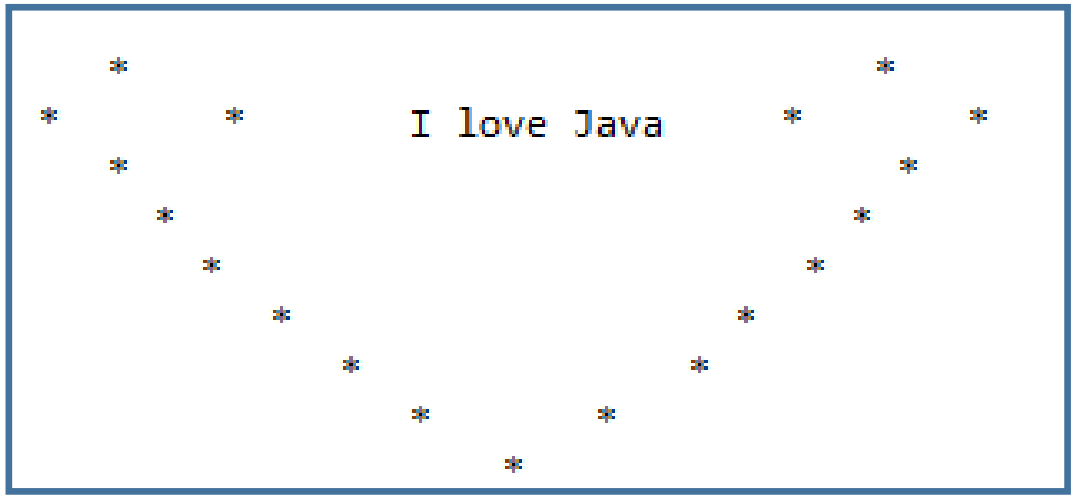
回顾

介绍了 Java 的跨平台性为代表的主要特征，然后介绍了如何下载和安装 Sun 公司提供的 Java 开发工具 JDK 以及使用说明。有了这些简单 Java 知识，就可以开发简单的 Java 程序。给出了这些程序从编辑源到用 Java 虚拟机执行的完整过程。

给出的实例实际有两种方式输入信息：通过命令行参数输入信息和在程序中通过键盘读入信息。然后经过简单加工，再通过标准输出显示到屏幕上。有了本章的基础，应该可以编写简单的程序了。

作 业

1. **调试本章 java 示例程序，观察结果**
2. 自己独立编写 HelloJava 程序，并配上必要的注释。
3. 将个人的基本信息（姓名、性别、籍贯、住址）打印到控制台上输出。各条信息分别占一行。
4. 结合 \n(换行)，\t(制表符)，空格等在控制台打印出如右图所示的效果。



作业发邮件到：

2230652597@qq.com

注意：

1. 本次作业要求大家用文本编辑器完成，在 CMD（命令行方式）下编译并运行

2. 下载安装一个 JAVA IDE（集成开发环境，图形方式），并熟悉常见功能

Java IDE 选择

图形化的IDE

- **1. IntelliJ IDEA**
- IntelliJ IDEA 是目前使用的最流行的 Java IDE 。它功能齐全，这意味着它有一长串提高生产率的功能，这使它成为许多 Java 开发人员的必备品。许多特性，比如代码完成，都是高度可定制的，并且可以调整以满足单个开发人员的需求。
- 它还具有高度的可扩展性，为库和插件提供了巨大的市场。
- 主要特点和好处：

➢ 智能代码完成：在编写代码时， IntelliJ IDEA 会在编写过程中提供建议。
➢ 导航到重复： IntelliJ IDEA 帮助用户轻松查找和删除重复的代码片段。
➢ 快速修复： IntelliJ IDEA 检测代码使用错误，在编辑器窗口中突出显示有问题的代码，并提供如何修复问题的说明。
➢ 版本控制：对于使用 GitHub 的开发人员， IntellijIDEA 允许用户在不离开 IDE 的情况下记录项目进度。
➢ 支持新的语言功能： IntelliJ IDEA 通常能够快速支持新的 Java 语言功能。

- IntelliJ IDEA 中的许多功能适合初学者 ----- 在校大学生注册后免费使用旗舰版。

图形化的IDE

IntelliJ IDEA

Coming in 2019.1 What's New Features Learn Buy Download

50% 0K/s 5.5K/s

Download IntelliJ IDEA

Windows macOS Linux

Ultimate

For web and enterprise development

DOWNLOAD .EXE

Free trial

Version: 2018.3.5
Build: 183.5912.21
Released: February 26, 2019
Release notes

System requirements
Installation Instructions

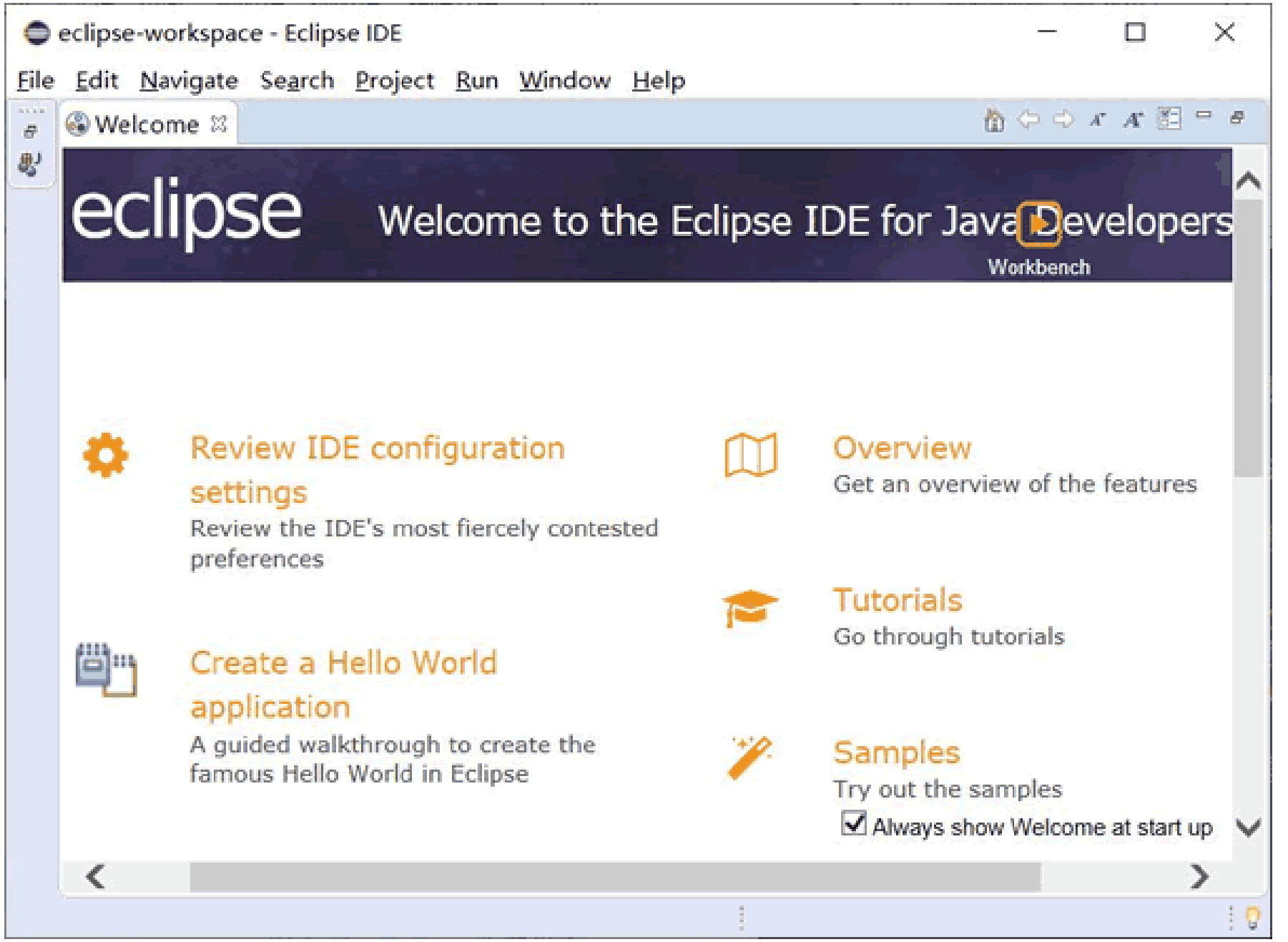
Cookies and IP addresses allow us to deliver and improve our web content and to provide you with a personalized experience. Our website uses cookies and collects your IP address for these purposes.

JetBrains may use cookies and my IP address to collect individual statistics and to provide me with personalized offers and ads subject to the Privacy Policy and the Terms of Use. JetBrains may use third-party services for this purpose. I can revoke my consent at any time by visiting the Opt-Out page.

[Y]es, I agree [N]o, thanks

图形化的IDE

- **2. Eclipse**
- Eclipse 和 IntelliJ IDEA 一样，是一个成熟的、功能齐全的 IDE ，它极大地改善了开发体验。这两种产品之间的关键区别主要集中在可用性 / 学习曲线和价格上。由于 IntelliJ IDEA 社区不具备 IntelliJ IDEA Ultimate 的全部功能，而且 Eclipse 在功能上更接近 IntelliJ IDEA ，因此免费产品之间的比较有利于 Eclipse 。然而，Eclipse 被认为比 IntelliJ IDEA 更难学习和使用，这意味着对于初学者来说可能很难。
- 特征
  - 高度可扩展性：从版本控制到编程语言， Eclipse 拥有一个大型的、经常更新的插件库。
  - 代码完成： Eclipse 提供代码完成功能，以及改进该功能的扩展。
  - 错误检测： Eclipse 还具有错误检测功能，并在检测到错误时提供更改建议。
  - 重构： Eclipse 提供了扩展的重构功能，使开发人员更容易重构代码。
- EclipseIDE 在经验丰富的开发人员中很受欢迎，并且比其他 IDE 更适合于大型项目。


eclipse-workspace - Eclipse IDE
File Edit Navigate Search Project Run Window Help
Welcome
eclipse
Welcome to the Eclipse IDE for Java Developers
Workbench
Review IDE configuration settings
Review the IDE's most fiercely contested preferences
Overview
Get an overview of the features
Tutorials
Go through tutorials
Create a Hello World application
A guided walkthrough to create the famous Hello World in Eclipse
Samples
Try out the samples
Always show Welcome at start up

图形化的IDE

图形化的IDE

- **3. NetBeans**
- NetBeans 是 Java 开发中使用的另一个免费 IDE 。虽然不像 Eclipse 或 IntelliJ IDEA 那样被广泛采用，但它确实有一小部分来自经验丰富的开发人员的专门追随者。
- 与 IntelliJ 和 Eclipse 一样， NetBeans 允许用户通过添加模块来扩展其功能。 NetBeans 的优点之一是它是 Apache 生态系统的一部分，这意味着为用户内置了 Apache Maven 功能。
- 特点和好处

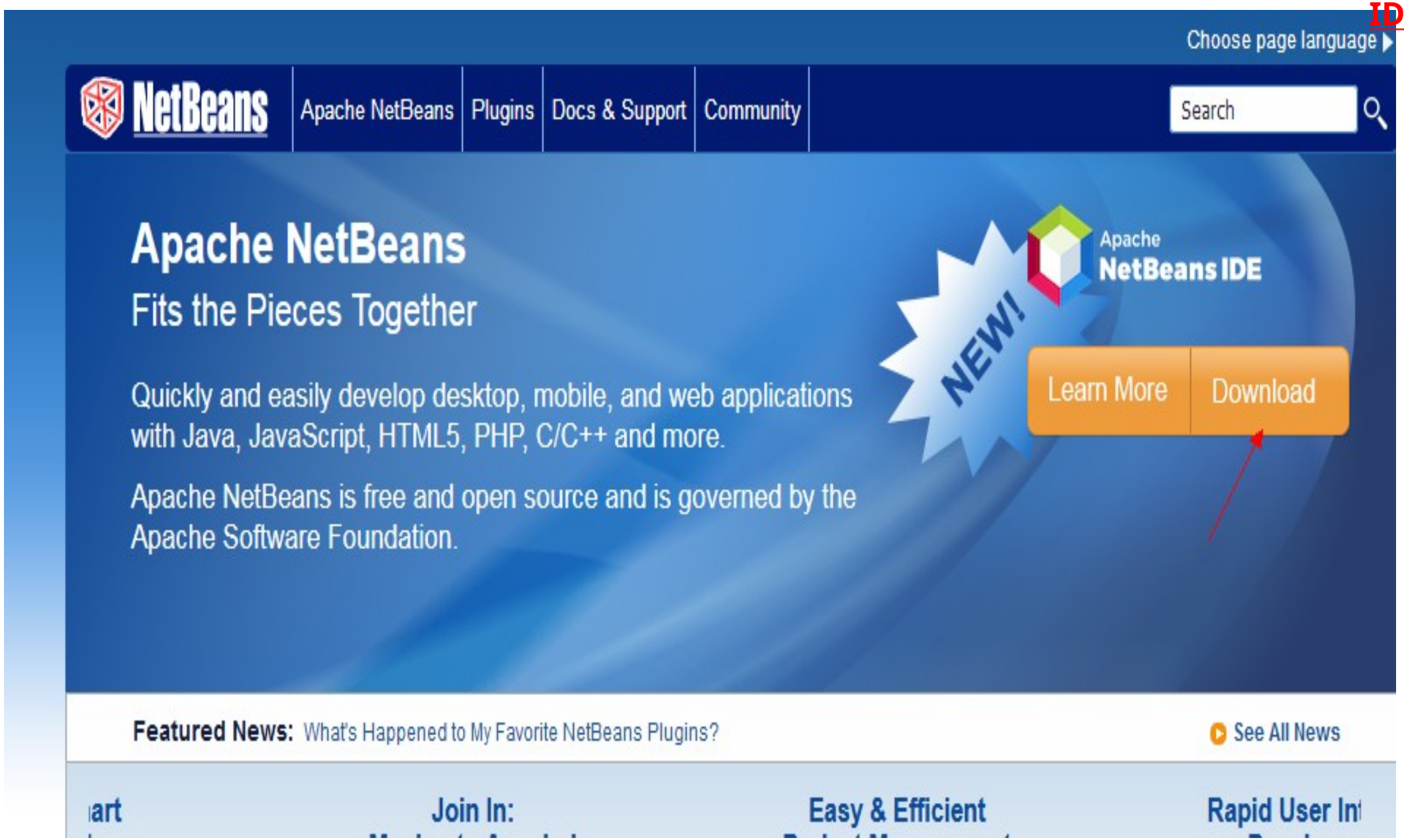
➢ Maven-Native ：因为 Maven 是 NetBeans 中的默认构建工具，所以它不需要安装，也不需要安装到本地 Maven 存储库。

➢ 轻量级： NetBeans 是轻量级的，但仍然提供现成的可靠功能。

➢ 可扩展性： NetBeans 是相当可扩展的，它提供了许多插件，可以帮助扩展 IDE 的功能。

- 想用 Maven 本地开发的开发人员会强烈地考虑 NetBeans 。它比 Eclipse 稍微容易使用，因此对于没有 IntelliJ IDEA 预算的初学者开发人员来说，它也是一个不错的选择。

图形化的IDE


Choose page language
NetBeans
Apache NetBeans
Plugins
Docs & Support
Community
Search
Apache NetBeans
Fits the Pieces Together
Quickly and easily develop desktop, mobile, and web applications with Java, JavaScript, HTML5, PHP, C/C++ and more.
Apache NetBeans is free and open source and is governed by the Apache Software Foundation.
NEW!
Apache NetBeans IDE
Learn More
Download
Featured News: What's Happened to My Favorite NetBeans Plugins?
See All News
art
Join In:
Easy & Efficient
Rapid User Int